Xu *et al. Cybersecurity* (2026) 9:230
https://doi.org/10.1186/s42400-026-00676-2

Cybersecurity

RESEARCH Open Access

SeqCube-HT: state-cube-guided generation of sequential hardware Trojan benchmarks

Yuhang Xu[1], Wei Guo[1*], Wenbo Zhang[1], Shuaikang Hou[1], Wenjian Zhang[1] and Zhihao Xu[1]

Abstract

Hardware Trojan (HT) detector evaluation depends on benchmarks whose triggers are executable from a defined reset state and whose payload effects reach an observation boundary. Establishing both properties is difficult for sequential Trojans because rare events are coupled through state evolution across cycles. We present SeqCube-HT, a state-cube graph framework that incorporates temporal execution evidence into benchmark construction. Reset-consistent traces are compressed into pre- and post-state cubes, and directed compatibility organizes candidate event sequences. Selected paths are then completed into concrete inputs and paired with sensitized payload paths. An instance is retained as witness-validated only after active and inactive replay and a bounded audit; the evaluated implementation materializes replay vectors, labels, logs, and audit records for that instance. Across four ISCAS'89 circuits and five Extended IP designs, SeqCube-HT produced 3240 witness-validated instances from 4500 attempts (72.0%). Under a matched protocol, the best-performing comparator implementation achieved 50.6% witness-validation yield and 39.5 s per witness-validated instance, compared with 72.0% and 20.9 s for SeqCube-HT. Covariate-matched detector experiments retained insertion-source differences under the evaluated graph probes. These results show that trace-backed temporal construction can efficiently produce witness-validated sequential HT benchmarks under the evaluated trace budget and recorded-execution validation protocol.

Keywords Hardware Trojan, Benchmark generation, Sequential circuit, State-cube graph, Hardware security

Introduction

Modern integrated circuits depend on distributed design and manufacturing flows that include third-party IP, EDA tools, fabrication, packaging, and testing services. This distribution improves productivity but also creates opportunities for malicious circuit modification (Tehranipoor and Koushanfar 2010; Bhunia et al. 2014; U.S. Department of Defense 2022). A hardware Trojan typically combines a trigger that waits for a rare logical or temporal condition with a payload that perturbs the design after activation (Rostami et al. 2014; Mishra et al. 2017). Because the modification is embedded in hardware, it may evade software defenses and is difficult to remove after fabrication (Hicks et al. 2010; Becker et al. 2013; Sturton et al. 2011).

Credible detector evaluation therefore depends on credible HT benchmarks. Trust-HUB and related efforts established a common basis for comparison (Shakya et al. 2017; Salmani et al. 2013), but benchmark scale and diversity do not by themselves ensure meaningful evaluation. The inserted trigger must be executable, and the payload effect must reach an observation boundary. An infected netlist that fails either condition may be syntactically well formed but can still misrepresent detector coverage. Reported unsatisfiable triggers, functional errors, and simulation mismatches in public instances make explicit execution and observation evidence necessary (Krieg 2023).

Automated insertion methods alleviate benchmark scarcity through rare-node sampling, feature-driven

*Correspondence:
Wei Guo
guowjss@126.com
[1] Institute of Information Technology, Information Engineering University (IEU), Zhengzhou 450001, China

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

selection, reinforcement-learning search, and compatibility-graph construction (Cruz et al. 2018, 2022; Gohil et al. 2022; Sarihi et al. 2024; Kumar et al. 2025). Large-netlist efforts have further emphasized labels and detector-stress evaluation (Popryho et al. 2026). Many flows, however, construct trigger and payload candidates before invoking simulation, ATPG, SAT, bounded model checking (BMC), or miter checks. These checks can reject candidates that lack a feasible execution or observable effect, but they do not prevent the generator from repeatedly proposing them.

The cost of late rejection is greater for sequential circuits. Whereas combinational compatibility is evaluated within one logic frame, a temporal trigger must follow a reset-reachable trajectory across multiple frames. This distinction is also reflected in the classical benchmark suites: ISCAS'85 contains combinational circuits, while ISCAS'89 includes sequential designs (Brglez and Fujiwara 1985; Brglez et al. 1989). Rare events must occur in the required order, and the payload must remain sensitized until its effect reaches a primary output or a state boundary. As Fig. 1 illustrates, treating these requirements only as post-construction filters increases failed attempts and leaves weak per-instance evidence. Formal methods and testability metrics remain valuable (Tiwari et al. 2009; Hu et al. 2014; Nahiyan et al. 2017; Goldstein and Thigpen 1980), but global symbolic checks are costly when placed in every candidate-generation loop.

SeqCube-HT addresses this bottleneck by using reset-consistent temporal evidence during candidate construction. It compresses observed transitions into state-cube events, connects locally compatible events in temporal order, and completes selected paths into concrete input sequences before insertion. Trigger constraints are then paired with payload-sensitization constraints. Active and inactive replay and a bounded audit determine witness validation, and the records retained for each evaluated instance support package-level replay and detector-label reconstruction.

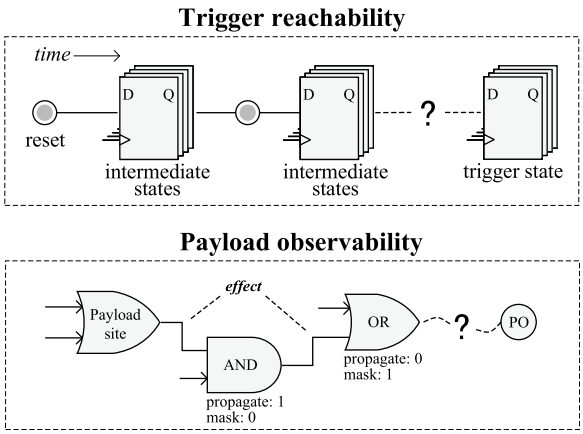


Fig. 1 Recorded-execution criteria for sequential HT construction. The retained trigger witness must execute from the stated reset model, and the payload effect must reach the recorded observation boundary

The main contributions of this paper are summarized as follows:

1. We formulate temporal-trigger construction over trace-backed state-cube events and a directed compatibility graph that preserves event order and reset-consistent provenance.
2. We complete candidate paths into concrete input sequences and merge trigger constraints with payload-sensitization constraints before insertion. For each witness-validated instance, the evaluated implementation retains active and inactive replays, labels, logs, and bounded-audit records.
3. We evaluate SeqCube-HT on four ISCAS'89 circuits and five Extended IP designs. A matched four-flow comparison measures witness-validation yield and cost, while covariate-matched and cross-scale detector studies characterize the resulting benchmark corpus.

The remainder of this paper is organized as follows. Section "Related work" reviews hardware Trojan benchmark generation and detection-related evaluation methods. Section "Methodology "presents the proposed SeqCube-HT methodology. Section "Experimental evaluation" reports the experimental evaluation. Section "Conclusion" concludes the paper.

Related work

HT benchmark-generation methods have advanced automation, feature modeling, search efficiency, and large-netlist labeling. For detector evaluation, the relevant distinction is not only how an infected design is generated, but also what evidence accompanies it. Table 1 summarizes capabilities and artifacts explicitly reported by representative methods. The entries describe documented support rather than rank benchmark quality or physical realism: ●denotes explicit support, ◐ denotes partial or indirect support, and ○ denotes unsupported or unreported support.

Trojan benchmarks

Trust-HUB and early trust benchmark efforts established a shared evaluation basis for HT research (Shakya et al. 2017; Salmani et al. 2013). Common host circuits and known Trojan instances make detector comparisons reproducible, but static collections grow slowly and cover only a fraction of the design space. Repeated tuning on

Table 1 Reported benchmark-generation capabilities and artifacts

Method	Automated generation	Temporal trigger	Trigger witness	Payload witness	Label export	Audit trail
Trust-HUB Shakya et al. (2017); Salmani et al. (2013)	○	◐	◐	◐	○	○
TRIT Cruz et al. (2018)	●	◐	◐	◐	○	○
MIMIC Cruz et al. (2022)	●	◐	◐	◐	○	○
Attrition Gohil et al. (2022)	●	○	○	○	○	○
Trojan Playground Sarihi et al. (2024)	●	○	○	○	○	○
CG Kumar et al. (2025)	●	○	◐	○	○	○
IndusHT Popryho et al. (2026)	●	◐	◐	◐	●	◐
SeqCube-HT	●	●	●	●	●	●

a small set of familiar structures can therefore produce optimistic estimates of detector coverage.

Benchmark validity is separate from benchmark availability. Krieg's analysis of Trust-HUB showed that detector evaluation requires a correct host, a meaningful modification, a reachable trigger, and an observable payload effect (Krieg 2023). For sequential Trojans, these properties must be evaluated under a stated reset model and a concrete temporal execution.

Conventional detection

Logic-testing methods seek input patterns that activate rare Trojan triggers and expose their functional effects. MERO, GA-SAT, TARMAC, and RL-TestGen pursue this objective through repeated rare-node excitation, satisfiability-guided optimization, clique-based sampling, or reward-guided vector generation Chakraborty et al. (2009); Saha et al. (2015); Lyu and Mishra (2021); Pan and Mishra (2021). MERS extends rare-event activation to side-channel-oriented test generation (Bhunia et al. 2016).

Side-channel detection instead observes power, delay, current, thermal, or electromagnetic signatures; MaxSense and NICE improve sensitivity in this setting (Lyu and Mishra 2021; Su et al. 2024). Although these detectors are outside the present evaluation, both logic testing and side-channel analysis depend on benchmark instances with meaningful activation and propagation behavior. Documented trigger execution and payload observation make a detector failure more interpretable than a result obtained from a syntactically modified but functionally invalid netlist.

Learning-based detection

Machine learning has been applied to both gate-level and RTL-level HT detection. Feature-based methods such as FANCI, RF-Trojan, and COTD use Boolean, structural, controllability, or observability attributes to identify suspicious logic (Waksman et al. 2013; Hasegawa et al. 2017; Salmani 2017). Graph-learning methods represent circuits as graphs and learn patterns for design-level detection, node-level localization, or multifunctional classification (Yasaei and Faruque 2025; Yasaei et al. 2021; Cheng et al. 2022, 2023; Lashen et al. 2023; Chen et al. 2025). Their conclusions, however, depend on the composition and evidential quality of the training and evaluation corpora.

The data requirement is therefore not simply a larger sample count. Learning-based detectors need graph-aligned labels, sufficient insertion diversity to limit memorization, and evidence that each positive instance has executable malicious behavior. Adversarial studies further show that logically equivalent or structurally modified Trojan implementations can mislead learned detectors (Nozawa et al. 2021). The evaluated corpus includes paired graph views, localization labels, and replay records for studying these effects.

Automatic Trojan insertion

Automatic insertion makes benchmark construction more configurable than manual design. TRIT controls trigger and payload insertion through rarity-oriented parameters (Cruz et al. 2018), while MIMIC augments candidate selection with functional and structural features learned from known Trojan populations (Cruz et al. 2022). Attrition and Trojan Playground formulate insertion as reinforcement-learning search against detection objectives (Gohil et al. 2022; Sarihi et al. 2024). These methods broaden the generated design space, but a retained trigger–payload pair still requires evidence of execution and observation.

Compatibility graphs reduce blind sampling by identifying rare nodes that can be activated together (Kumar et al. 2025). This constructive screening avoids many same-cycle conflicts before validation. Sequential generation requires a different relation:

candidate events must be ordered across reset-consistent state transitions and then completed into an executable sequence. SeqCube-HT uses state-cube compatibility for this temporal screening; replay and the bounded audit remain the final validation criteria.

Large-netlist work such as IndusHT adds scalable rarity mining, labels, and metadata for detector-stress studies (Popryho et al. 2026). The present work shares this benchmark-facing objective but focuses on temporal triggers in sequential netlists and on replayable trigger and payload evidence for each retained instance.

Physical insertion constraints

Physical and post-synthesis studies show that insertion feasibility also depends on foundry, reverse-engineering, FPGA, and layout constraints (Fyrbiak et al. 2019; Jyothi et al. 2017; Yu et al. 2019; Trippel et al. 2021; Zhang et al. 2014; Hepp et al. 2022; Perez and Pagliarini 2023; Trippel et al. 2020). SeqCube-HT instead studies functional benchmark generation for single-clock gate-level netlists; Section "Problem formulation" states the corresponding scope.

Methodology

SeqCube-HT takes a sequential gate-level netlist, a reset model, and a generation budget as input and returns paired clean/infected netlists with replay and audit records. Figure 2 summarizes the four-stage construction and witness-validation workflow. Stage 1 mines rare events from reset-consistent traces, and Stage 2 abstracts each observed transition into paired pre- and post-state cubes. Stage 3 constructs the directed state-cube transition graph and selects a reset-anchored candidate path, which is then completed into concrete inputs. Stage 4 couples the completed trigger path with a payload witness, realizes the modification, and applies active and inactive replay and the bounded audit before packaging. The graph therefore screens local temporal inconsistencies rather than proving global reachability; concrete completion and replay determine executability. The inserted Trojans are additive gate-level modifications intended for detector evaluation (Cruz et al. 2018, 2022; Gohil et al. 2022; Kumar et al. 2025).

Problem formulation

Given a single-clock gate-level sequential netlist N, a reset model r, and a generation budget B, SeqCube-HT operates under functional simulation semantics and

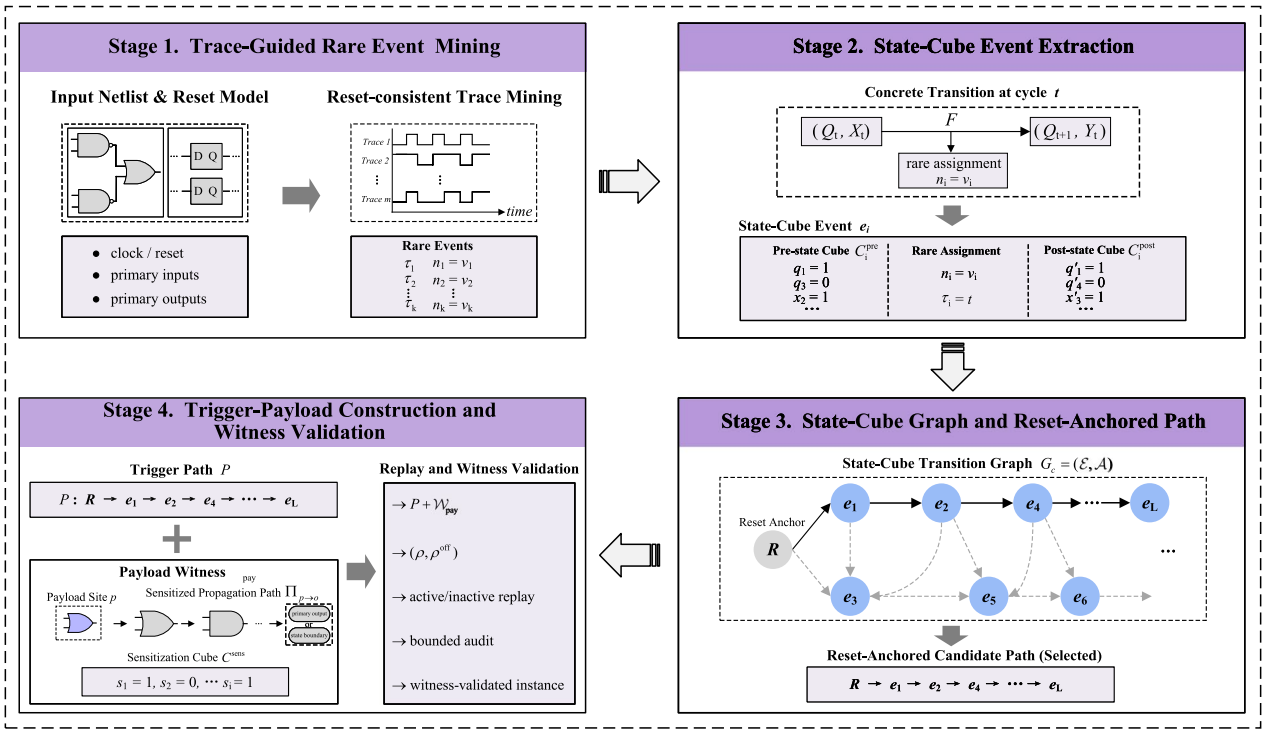


Fig. 2 Four-stage SeqCube-HT workflow. Reset-consistent traces yield rare transition events; paired state cubes form a directed graph for reset-anchored path selection; the completed trigger path is then coupled with a payload witness and subjected to active and inactive replay and the bounded audit before it becomes a witness-validated instance

produces a package set $\mathcal{D} = \{I_1, \ldots, I_m\}$. Each retained instance contains paired netlists, trigger and payload metadata, detector labels, replay vectors, and an audit record. The formulation has three goals: use trace-backed temporal consistency before insertion, couple payload observation with trigger selection, and export the evidence needed to reproduce validation. Let $A_T(\rho)$ indicate that replay sequence ρ executes the selected trigger path, let $\Delta_o(\rho)$ indicate a clean/infected difference at observation boundary o, and let $\mathcal{B}_d(\rho, \rho^{\text{off}}, o)$ denote the bounded-audit result at depth d. An instance is retained as witness-validated only when

$$\begin{aligned} &A_T(\rho) = 1, \quad \Delta_o(\rho) = 1, \quad \Delta_o(\rho^{\text{off}}) = 0, \\ &\mathcal{B}_d(\rho, \rho^{\text{off}}, o) = 1. \end{aligned} \tag{1}$$

The active replay executes the selected trigger and exposes the payload. Under the fixed inactive sequence, no clean/infected difference may be observed at the same boundary, and the final term audits both recorded executions to depth d.

For the symbolic audit, let $o = (z_o, t_o)$ identify the recorded primary-output or state signal z_o and its observation cycle $t_o \le d$. Let $\mathcal{U}_d(N, N', r, u)$ be the joint d-cycle unrolling of the clean netlist N and infected netlist N' from reset model r, with their primary inputs tied to the recorded sequence u through depth d. The stored sequence includes the observation-extension inputs. The checker constructs

$$\Phi_{\text{on}} = \mathcal{U}_d(N, N', r, \rho) \wedge [A_T(\rho) = 1] \wedge [z_o^N(t_o) \neq z_o^{N'}(t_o)], \tag{2}$$

$$\Phi_{\text{off}} = \mathcal{U}_d(N, N', r, \rho^{\text{off}}) \wedge [z_o^N(t_o) \neq z_o^{N'}(t_o)]. \tag{3}$$

The bounded audit passes exactly when

$$\begin{aligned} &\mathcal{B}_d(\rho, \rho^{\text{off}}, o) = 1 \iff \text{status}(\Phi_{\text{on}}) = \text{sat} \\ &\wedge\ \text{status}(\Phi_{\text{off}}) = \text{unsat}. \end{aligned} \tag{4}$$

A timeout, an unknown result, or either opposite status causes rejection. The SAT result for Φ_{on} confirms the recorded activating execution and output difference; the UNSAT result for Φ_{off} excludes a difference at the same boundary for the fixed inactive sequence. These queries do not quantify over other inputs or establish unbounded reachability or non-interference.

Definition 1 A SeqCube-HT witness package contains the paired netlists, reset model, active and inactive input sequences through depth d, observation boundary, simulator configuration, trigger and payload metadata, and the two bounded-audit query records (encoding, depth, timeout, and solver status). An independent checker replays both sequences, compares the recorded boundary values, reconstructs Φ_{on} and Φ_{off}, and verifies the SAT/UNSAT outcomes in Eq. (4). A passing package establishes Eq. (1) only for these recorded inputs and settings.

Throughout this paper, a *candidate path* denotes graph output before concrete completion, *replay-accepted* denotes agreement with the recorded active/inactive sequence pair, and a *witness-validated instance* denotes an instance satisfying Eq. (1) under the recorded reset model, observation boundary, audit depth, simulator, and solver settings. The corresponding attempt-level metric is *witness-validation yield*, namely the fraction of attempts that produce witness-validated instances. Witness validation does not denote complete reachability, unbounded non-interference, or correctness for unrecorded inputs.

Evidence scope Trace records and state-cube edges support candidate construction rather than global reachability: a trace record confirms one reset-consistent transition, and a $g = 1$ edge excludes only a direct endpoint conflict; for $g > 1$, the intermediate transitions remain unresolved. A graph path is therefore a locally screened candidate until concrete completion. Active replay then establishes trigger execution and a payload difference at the recorded boundary, while the paired inactive replay excludes that difference only for one matched negative sequence. The bounded audit verifies these two fixed executions under the recorded reset, boundary, depth, and solver semantics; it does not establish unbounded reachability or non-interference.

Scope and limitations Beyond these witness-level boundaries, the functional model does not cover synthesis preservation, timing closure, placement and routing, power, congestion, scan reconfiguration, clock-domain crossings, multiple reset domains, analog effects, or foundry constraints. Consequently, the larger-IP experiments assess computational feasibility on the evaluated gate-level netlists rather than physical stealth or deployment feasibility.

Algorithm 1 summarizes the generation loop. Graph search proposes temporally screened paths; concrete completion, trigger–payload construction, replay, and the bounded audit decide whether a path yields a witness-validated instance.

Algorithm 1 SeqCube-HT generation and witness validation

Require: Gate-level netlist N, reset model r, generation budget B, trigger length L
Ensure: Benchmark package set $\mathcal{D}$

$\bar{N} \leftarrow \textsc{Normalize}(N)$
$\mathcal{T} \leftarrow \textsc{Trace}(\bar{N}, r)$
$\mathcal{W} \leftarrow \textsc{MineWitnesses}(\mathcal{T})$
$\mathcal{E} \leftarrow \textsc{ExtractEvents}(\mathcal{W})$
$G_c \leftarrow \textsc{BuildGraph}(\mathcal{E})$
$\mathcal{D} \leftarrow \varnothing$
for $k = 1, \ldots, B$ **do**
  $P \leftarrow \textsc{SelectPath}(G_c, L)$
  $c \leftarrow \textsc{Complete}(P, r)$
  if $c = \textsc{Fail}$ **then**
    continue
  end if
  $(\rho, \rho^{\text{off}}) \leftarrow c$
  $\mathcal{W}_{\text{pay}} \leftarrow \textsc{ConstructPayload}(\bar{N}, P, \rho)$
  if not $\textsc{Compatible}(P, \mathcal{W}_{\text{pay}})$ **then**
    continue
  end if
  $N' \leftarrow \textsc{Insert}(\bar{N}, P, \mathcal{W}_{\text{pay}})$
  if not $\textsc{ReplayAccepts}(\bar{N}, N', \rho, \rho^{\text{off}})$ **then**
    continue
  end if
  $\eta \leftarrow (P, \mathcal{W}_{\text{pay}}, \rho, \rho^{\text{off}})$
  $a \leftarrow \textsc{Audit}(\bar{N}, N', \eta)$
  if $a = \textsc{Pass}$ **then**
    $I \leftarrow \textsc{Package}(\bar{N}, N', \eta, a)$
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{I\}$
  end if
end for
return $\mathcal{D}$

State-cube extraction

Stages 1 and 2 of Fig. 2 mine reset-consistent rare transitions and abstract each retained witness into paired state cubes. To define the transition context, SeqCube-HT first cuts the registers at the current and next time steps to form the one-cycle frame in Eq. (5):

$$\mathcal{F} : (Q_t, X_t) \mapsto (Q_{t+1}, Y_t). \tag{5}$$

Here, Q_t is the current register state, X_t is the primary-input vector, Q_{t+1} is the next register state, and Y_t is the output vector. The reset model initializes every simulated trace, attaching each mined event to an observed execution rather than an arbitrary symbolic state.

At each simulated cycle, the method observes internal values and retains low-probability events on non-control nets. Signal probability and toggle behavior identify low-activity regions; structural depth, fanout, and testability information exclude shallow or intrusive sites (Cruz et al. 2022; Popryho et al. 2026; Goldstein and Thigpen 1980). A transition witness records the event $n_i = v_i$, its current-state and input context, the observed next state and outputs, and the source trace/cycle identifier τ_i. Events near clock, reset, scan, or test-control logic are removed to prevent the generator from exploiting nonfunctional control points.

State context distinguishes trace-backed events from isolated rare values (Fig. 3). Each value is stored with the current state, input, next state, and output information that made the observation replayable. Two nodes with similar empirical probabilities can therefore be treated differently when only one occurs on a reset-consistent transition.

Rarity is evaluated before cube extraction so that graph construction does not spend its budget on common events. For an internal event $n = v$, let $\hat{p}(n = v)$ be its empirical frequency in the reset-consistent trace set. SeqCube-HT uses

$$R(n, v) = -\log_{10}\big(\max(\hat{p}(n = v), \epsilon)\big), \tag{6}$$

where ϵ prevents a zero-frequency singularity. Equation (6) ranks trace-supported low-frequency events; filters on depth, fanout, duplicated signatures, and clock/reset/scan/test overlap are then applied to the ranked pool.

Because a concrete trace snapshot fixes many literals unrelated to the rare event, SeqCube-HT compresses each witness into a pair of state cubes. A state cube is a partial Boolean assignment,

$$C = \{(z, b) : z \in \mathcal{Z},\ b \in \{0, 1\}\}, \tag{7}$$

where $\mathcal{Z}$ denotes state and signal variables and only retained literals appear in C. The pre-cube C_i^{pre} is defined over the current state and inputs (Q_t, X_t), whereas the post-cube C_i^{post} is defined over next-state registers Q_{t+1}. Together with the rare event and trace identifier, the two cubes define

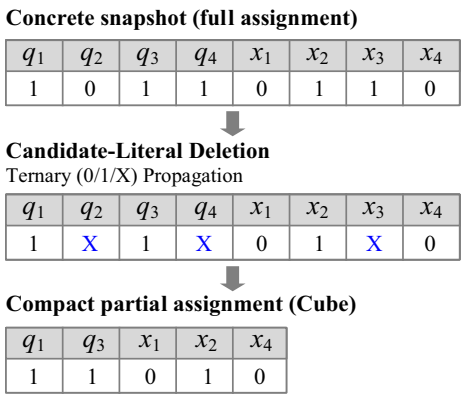
$$e_i = (n_i, v_i, C_i^{\text{pre}}, C_i^{\text{post}}, \tau_i). \tag{8}$$

The retained trace identifier provides a concrete starting point when a symbolic path is later completed into an executable input sequence.

Cube minimization starts from the full trace snapshot and removes one candidate pre-cube literal at a time (Fig. 4). Ternary propagation re-evaluates the affected logic, and a literal is discarded only if $n_i = v_i$ remains stable. Known next-state values form the post-cube; concrete trace completion supplies missing values when needed. Because literal order can change the resulting non-minimum cube, the implementation uses a deterministic order for each configuration and records it with the seed and netlist hash. Table 2 reports the experimental order.

Each retained event carries its rarity score, structural metadata, reset-anchor status, and trace snapshot into the transition graph. Events with excessive fanout disturbance, unstable reset behavior, duplicated signatures, or overlap with clock and test-control structures are discarded.

Trace-conditioned search scope Trace conditioning is deliberate: SeqCube-HT uses observed events to reduce late replay failures during benchmark construction, not to approximate reachable-state coverage. Every graph node originates from an event observed under the declared



q_1	q_2	q_3	q_4	x_1	x_2	x_3	x_4
1	0	1	1	0	1	1	0

q_1	q_2	q_3	q_4	x_1	x_2	x_3	x_4
1	X	1	X	0	1	X	0

q_1	q_3	x_1	x_2	x_4
1	1	0	1	0

Fig. 4 Pre-state cube extraction by deterministic literal deletion and ternary propagation

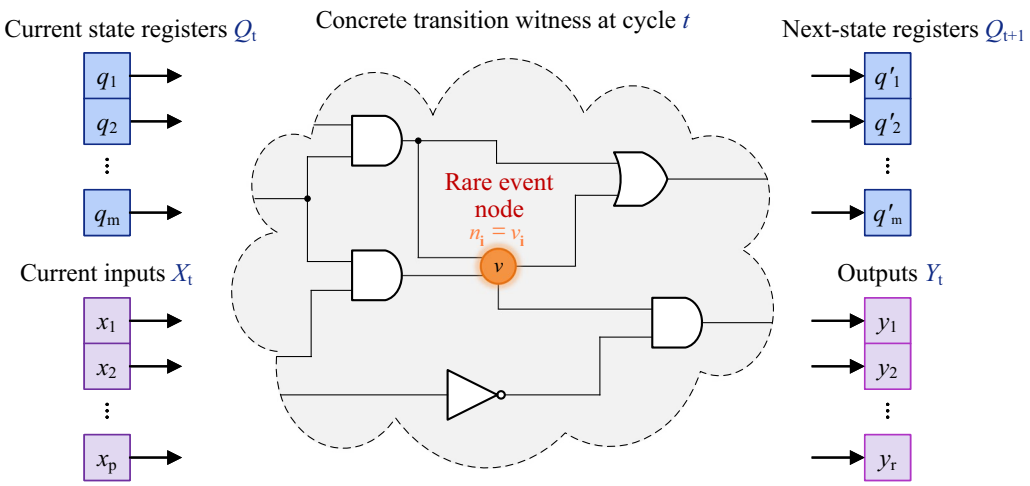


Fig. 3 Trace-backed state-transition witness

reset model, trace budget, stimulus distribution, and random seed. Concrete completion may connect events taken from different traces and traverse intermediate states that were not retained as graph nodes, but a trigger event absent from the mined pool cannot enter a candidate path. Path diversity and duplicate suppression therefore operate within that pool. The resulting candidates are more likely to admit recorded replay witnesses, at the cost of missing reachable events that are rare or absent under the sampled traces and that a symbolic exploration might find.

Algorithm 2 State-cube event extraction

Require: Rare transition witness w_i
Ensure: State-cube event e_i or rejection reason
$C^{\mathrm{pre}} \leftarrow \textsc{SupportCube}(w_i)$
for each literal ℓ in C^{pre} **do**
  $\widetilde{C} \leftarrow C^{\mathrm{pre}} \setminus \{\ell\}$
  if $\textsc{TernaryStable}(w_i, \widetilde{C}, n_i = v_i)$ **then**
    $C^{\mathrm{pre}} \leftarrow \widetilde{C}$
  end if
end for
$C^{\mathrm{post}} \leftarrow \textsc{KnownPostCube}(w_i)$
if $C^{\mathrm{post}} = \varnothing$ and completion is enabled **then**
  $C^{\mathrm{post}} \leftarrow \textsc{SnapshotPostCube}(w_i)$
end if
if not $\textsc{QualityAccepts}(C^{\mathrm{pre}}, C^{\mathrm{post}})$ **then**
  return rejection reason
end if
return $e_i = (n_i, v_i, C^{\mathrm{pre}}, C^{\mathrm{post}}, \tau_i)$

Algorithm 2 balances two competing requirements. Retaining too many literals creates artificial conflicts during temporal stitching, whereas removing too many increases the risk of replay mismatch. The extracted cube therefore preserves event stability and next-state consistency with a compact set of literals.

Trigger-path synthesis

Stage 3 of Fig. 2 converts the extracted events into a directed transition graph for reset-anchored candidate-path selection. Viewing each cube as a partial map, two cubes are compatible when they agree on their shared variables:

$$C_a \bowtie C_b \iff \forall z \in \mathrm{dom}(\mathrm{C_a}) \cap \mathrm{dom}(\mathrm{C_b}), \; C_a(z) = C_b(z). \tag{9}$$

Because C_i^{post} is expressed over next-state variables and C_j^{pre} is expressed over current-state variables and inputs, the comparison requires an explicit alignment. Let $\sigma(C_i^{\mathrm{post}})$ rename each retained q_{t+1} literal to the corresponding current-state variable q_t, let $\pi_Q(C_j^{\mathrm{pre}})$ restrict the pre-cube to its state literals, and let $\mathcal{G}_{ij} \subseteq \{1, \ldots, g_{\max}\}$ contain the candidate separations allowed by the trace-order and reset-window filters. SeqCube-HT constructs the locally compatible edge set

$$\mathcal{A} = \{(e_i, e_j, g) : \sigma(C_i^{\mathrm{post}}) \bowtie \pi_Q(C_j^{\mathrm{pre}}),\ g \in \mathcal{G}_{ij}\}. \tag{10}$$

The tuple retains the temporal gap and trace provenance used during completion. For $g = 1$, the test excludes a direct conflict between produced and enabling state literals. For $g > 1$, it screens only the two endpoints; the intermediate $g - 1$ transitions are resolved during concrete sequence completion. The edge is directional because produced and enabling states are not interchangeable.

Unlike same-cycle compatibility among rare nodes (Kumar et al. 2025), the edge in Fig. 5 relates a produced state to the enabling state of a later event. A selected trigger path

$$P = (e_1, e_2, \ldots, e_L) \tag{11}$$

begins at a reset-compatible pre-cube and follows the directed edges of G_c. Concrete completion and replay determine whether this locally screened path admits a reset-consistent execution under the recorded settings.

Path selection uses beam search over G_c. Candidate paths are ranked by rarity, temporal span, diversity, and implementation cost:

$$S(P) = R(P) + D(P) - H(P), \tag{12}$$

where $R(P)$ aggregates event rarity, $D(P)$ rewards temporal progress and cone diversity, and $H(P)$ penalizes retained literals and trigger-monitor cost. The score in Eq. (12) is used only for ranking; paths with repeated signatures, excessive overlap, or no replay completion are discarded. The ranking favors paths that are temporally coherent, nontrivial for detector evaluation, and not dominated by a large inserted monitor.

Graph pruning and completion Before beam search, SeqCube-HT rejects candidate edges that lack reset-window or trace-order support, contain conflicting aligned literals, provide insufficient shared-state support, or exceed the trigger-logic budget. Events are indexed by reset-anchor support, rarity bucket, trace order, and overlapping state variables so that compatibility is evaluated only for pairs that can plausibly occur in temporal order, avoiding a dense all-pairs graph on large designs. Structural diversity and implementation cost enter the path score in Eq. (12) rather than acting as separate feasibility guarantees. Concrete completion occurs after path selection; a path without a reset-consistent input sequence is recorded as a completion failure rather than a pruning failure. This distinction prevents local graph consistency from being interpreted as path feasibility and supports circuit-level diagnostics when larger designs expose more rare events without proportionally more executable combinations.

Trigger–payload witness construction

Stage 4 of Fig. 2 couples a completed trigger path with payload-observation constraints and applies the replay and bounded-audit checks before packaging. A reset-anchored trigger path defines when the Trojan can fire; a payload witness determines whether its effect can be observed. Before modifying the netlist, the generator searches from candidate site p for a sensitized path $\Pi_{p\to o}$ to observation boundary o. The boundary is either a primary output or a state signal whose effect becomes externally visible after additional cycles. The resulting witness is

$$\mathcal{W}_{\text{pay}} = (p, o, \Pi_{p\to o}, C^{\text{sens}}, \delta), \tag{13}$$

where C^{sens} records the side-input assignments required to propagate payload effect δ. Local propagation rules, dominance information, and testability-guided ranking identify candidate witnesses (Salmani 2017; Tiwari et al. 2009; Hu et al. 2014; Nahiyan et al. 2017).

Trigger and payload constraints are merged only when their phase-local assignments are compatible:

$$C_P \bowtie C^{\text{sens}}. \tag{14}$$

Here, C_P denotes the trigger-path cube at the activation phase. A conflict causes the generator to change the payload site or propagation path, or to reject the attempt. A successful merge records the trigger path, payload witness, side conditions, and replay sequence as separate components so that later checks can be rerun independently.

After merging the trigger and payload constraints, SeqCube-HT realizes both components in the target netlist. Trigger implementation follows a low-overhead priority order: reuse existing state signals when possible, introduce shared monitors when reuse is insufficient, and add sticky logic only when temporal accumulation is required. Payload templates are limited to replay-checkable perturbations, including guarded bit flips, mux-based overrides, and state-boundary modifications.

Because the artifacts are intended for detector evaluation, insertion is restricted to replay-checkable payloads and bounded added logic. Trigger logic is added only after a selected event path has a replay sequence, and payload logic is placed only at sites with propagation

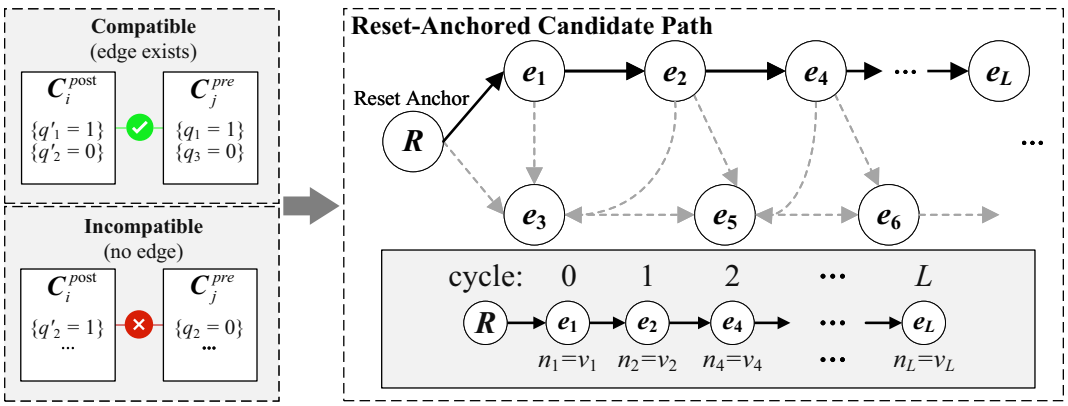


Fig. 5 State-cube graph construction and candidate-path selection

evidence. Among feasible sites, the ranking favors lower added-cell cost, excludes clock/reset/test logic, and retains both primary-output and state-boundary observations. These constraints limit trivial structural artifacts while preserving auditability.

Replay is mandatory. Sequence ρ is applied from the declared reset state to both netlists and must reproduce trigger activation and payload observation under identical simulator semantics. The paired sequence ρ^{off} changes one trigger-critical condition while preserving reset, replay length, and observation boundary; no clean/infected difference may be observed at that boundary under this fixed sequence.

The bounded symbolic audit in Eqs. (2)–(4) follows replay-accepted construction. The active unrolling must satisfy the scheduled trigger phases and recorded clean/infected difference; the inactive difference query must be UNSAT. The package records both encodings, expected and actual statuses, depth, timeout, and solver version. Every retained experimental instance has passed this audit. Section "Matched efficiency comparison" separately evaluates comparators that place symbolic checks inside the candidate loop (Tiwari et al. 2009; Hu et al. 2014; Nahiyan et al. 2017; Zhang et al. 2015).

Benchmark packaging

For every witness-validated instance, the evaluated implementation materializes a bounded replay witness package containing the paired netlists, trigger and payload metadata, active and inactive vectors through the audit depth, simulator logs, node labels, audit encodings and statuses, configuration, seed, tool version, and netlist hash. Netlists, vectors, labels, logs, configurations, and verification scripts occupy separate directories; rights and third-party provenance are recorded separately. The internal checker replays both vectors, compares boundary values and hashes, reconstructs Φ_{on} and Φ_{off}, checks their statuses, and derives labels from the clean/infected structural difference. Redistribution of source designs remains subject to their applicable permissions (Popryho et al. 2026).

Artifact scope and availability The directory structure above describes records materialized internally for witness-validated experimental instances; it does not imply that the full corpus is public. Instead, a self-contained, non-proprietary witness-validation reference artifact (v1.0) is publicly available; it is built around an author-created sequential controller/datapath. Its ten-cycle active sequence reaches three state-dependent trigger phases at cycles 2, 4, and 6; two registered payload stages then produce a primary-output difference at cycle 8. The matched inactive sequence changes only ack at cycle 4 and prevents the second phase while preserving reset and the observation boundary.

The archive contains the paired structural netlists, both vectors, trigger and payload descriptions, state-cube and path metadata, complete node labels, SMT-LIB records, configuration and logs, hashes, a manifest, and a verification script. The verification code under verify/is licensed under MIT; the author-created reference design, vectors, metadata, audit records, and logs are licensed under CC BY 4.0. The script parses the netlists, replays both sequences, checks the cubes, gaps, labels, and hashes, independently rebuilds the active-SAT and inactive-UNSAT bounded queries, and can invoke an external Verilog replay.

The reference artifact makes the schema and verification path independently inspectable, but it is not a member of the reported 3,240-instance corpus and cannot reproduce the generation-efficiency or detector experiments. The complete generator, experimental configurations and seeds, corpus, and per-instance packages remain restricted because the generator and run records belong to a project-internal toolchain that the authors are not authorized to redistribute, and several source and derived netlists carry third-party licenses. The article therefore makes no claim of public replay for the reported corpus.

Runtime is reported for reset-consistent simulation, cube extraction and minimization, graph construction, path search, payload-witness construction, insertion, replay, and audit. Rarity buckets, state-variable overlap, and fanout context prune pairwise compatibility, avoiding an unfiltered comparison of all trace events in large designs.

For every attempt, SeqCube-HT logs the first failed construction stage. The experimental evaluation can therefore relate retained instances to the search effort and late-stage failures that produced the observed yield.

The dominant cost changes with circuit scale. Let $F_{\mathcal{T}}$ be the number of simulated frames, $|\mathcal{E}|$ the number of filtered events, $\bar{d}$ the average retained cube size, b the beam width, and L the trigger length. Trace mining is linear in the evaluated frames and nets, while cube extraction scales with retained-literal checks and affected-cone propagation. Graph construction depends on the pruned compatibility candidates rather than all event pairs. After graph construction, beam search is approximately $O(bL\bar{d})$, and replay scales with sequence length and simulation cost. Section "Generation yield, witness validation, and cost" measures these stages separately.

The default policy combines bounded temporal span, beam-limited path search, deterministic ternary minimization, ranked payload search, mandatory replay, and a

post-generation bounded audit. Parameters are not tuned separately to maximize each circuit's witness-validated count. Both suites use the same parameter family, with only predeclared size-dependent resource limits; Table 2 lists the principal settings.

Experimental evaluation

Experimental setup

SeqCube-HT was implemented in Python 3.10 using NumPy, NetworkX, PyYAML, and Z3; the graph-detector probes use PyTorch and PyTorch Geometric. The evaluation covers four ISCAS'89 sequential circuits (s1423, s13207, s15850, and s35932) and five Extended IP designs (RS232, PIC16F84, wb_conmax, AES, and EthMAC10GE). Their flip-flop and gate counts appear in Table 3. Unless stated otherwise, generation and timing experiments ran on Ubuntu 22.04 with an Intel Core i9-13900K CPU and 64 GB RAM. Graph-model training and inference used an NVIDIA GeForce RTX 3090 GPU with 24 GB memory.

The evaluation addresses four questions. First, how many witness-validated instances can SeqCube-HT produce under a fixed attempt budget, and at what cost? Second, does temporal screening improve witness-validation yield and amortized runtime under a matched four-flow protocol? Third, how sensitive are the results to the principal generation parameters? Finally, what structural evidence do fixed graph-detector probes observe before and after covariate matching and ISCAS-to-IP transfer?

Each detector instance comprises one infected graph and its paired clean graph, giving a 1:1 class balance within every circuit, insertion source, and split stratum. We report infected-class precision and recall, clean-class specificity, graph-level F1 and PR-AUC, node-level localization precision and recall, and 95% confidence intervals. Because instances from the same host share most of their topology, the instance-random split is interpreted as a within-host upper bound. Circuit-held-out, source-held-out, and cross-scale experiments provide stricter transfer tests.

All generation tables use attempted instances as the denominator and apply Eq. (1) as the witness-validation gate.

For resetless ISCAS'89 netlists, all-zero DFF initialization is an explicit analysis convention rather than a claimed native reset; traces containing unresolved X values are rejected. The two size-dependent trace allocations use the same total frame budget, and nonfunctional controls remain in their declared normal mode. The inactive replay changes one trigger-critical condition while preserving reset, length, and observation boundary. An audit timeout or unknown result is classified as rejection.

Table 2 Main experimental configuration

Parameter	Setting
Reset/initialization	Native reset for three cycles; all-zero DFF convention for resetless netlists
Trace budget	131072 frames (1024×128 or 512×256)
Normal-mode trace stimulus	Independent Bernoulli(0.5) draws on functional primary inputs; nonfunctional controls fixed
Event-frequency threshold	$\widehat{p} \leq 10^{-2}$
Toggle-rate threshold	≤ 0.02
Rarity-score floor	$\epsilon = 10^{-6}$
Cube-minimization order	PI literals, then state literals; increasing cone size; net-index tie break
Trigger length	$L = 3$–6 with probabilities 0.15/0.35/0.35/0.15
Interphase gap	1–4 cycles
Beam width	$b = 64$
Shared-state support	At least two literals or 20% of the smaller cube
Duplicate suppression	Jaccard similarity > 0.90
Replay limit	24 or 32 cycles, selected by design size
Observation extension	8 cycles
Audit depth	$d = \min(40, \lvert\rho\rvert + 8)$
Audit timeout	60 s; timeout or unknown is rejected
Inner-loop symbolic timeout	30 s per CF-SAT/BMC candidate query; timeout or unknown is rejected
Generation budget	500 attempts per design
Detector repeats	10 seeds
Detector split	70/15/15 train/validation/test
Class balance	1:1 clean/infected

Statistical analysis Descriptive yield is an attempt-level proportion with a Wilson interval conditional on the nine evaluated designs; the interval does not treat those designs as a random circuit population. For between-flow comparisons, the inferential unit is a complete circuit–seed run containing all 500 attempts because attempts within a run share the host, event inventory, and search state. Runs are paired by circuit and seed. The prespecified effect sizes are the paired difference in witness-validation yield and the ratio of amortized seconds per witness-validated instance. Their 95% confidence intervals use 10,000 paired block-bootstrap resamples of complete runs, with the runtime ratio recomputed from total elapsed time and witness-validated count. Two-sided paired permutation tests are applied to run-level yield differences and log runtime ratios. The three SeqCube-HT-versus-comparator contrasts form one Holm-corrected family for each primary endpoint. Peak memory, solver calls, rejection stages, and timeout counts are secondary descriptive diagnostics.

For detector comparisons, the statistical unit is one seeded train/validation/test run after matching has been fixed without detector outcomes. Each run contributes one prespecified circuit-macro, backbone-macro score per source. For source a and SeqCube-HT, seed s contributes $D_s = M_{a,s} - M_{\mathrm{SeqCube},s}$. We report a two-sided Student interval over the ten differences and use a two-sided paired t-test. The three source contrasts are Holm-corrected separately for graph-level F1 and localization recall. These intervals quantify split and optimization variability on the fixed corpus, not uncertainty over unseen circuit populations. Class-wise intervals likewise use the ten seed-level macro scores rather than treating graphs or nodes as independent observations.

Generation yield, witness validation, and cost

The first experiment asks how many witness-validated instances can be obtained without making successful validation the stopping criterion. Each ISCAS'89 and Extended IP design receives 500 attempts. SeqCube-HT produces 3240 witness-validated instances from 4500 attempts, for an overall yield of 72.0% (95% Wilson CI: 70.7–73.3%). Yield is 77.2% on ISCAS'89 (75.3–79.0%) and 67.8% on the Extended IP designs (66.0–69.6%). Every retained instance passes replay and the post-generation audit. Section "Matched efficiency comparison" compares the same witness-validation criterion across four matched flows.

The rejection profile is circuit dependent rather than dominated by one universal failure mode (Fig. 6). Trigger-path selection, cube compatibility, and payload-witness construction account for most rejected attempts. Smaller designs exhibit more payload-site conflicts, whereas the larger IP designs produce more timeouts and replay mismatches. Recording the first failed stage exposes these differences without conditioning the analysis on retained instances.

Yield must be interpreted with the cost of obtaining each witness-validated instance. We profile 100 such instances per design; the ISCAS'89 subset also supplies the SeqCube-HT pool for the later matched detector experiment. Table 3 reports the median and interquartile range in seconds per witness-validated instance, with post-replay audit time shown separately. Runtime increases with circuit scale: the ISCAS'89 designs span 5.86–18.63 s, while the three largest Extended IPs require 24.76–43.87 s.

Stage-level timing localizes the generation cost to rare-event mining, trigger-path construction, payload-witness construction, insertion, and replay (Fig. 7). The bounded audit contributes a separate post-replay cost rather than an inner-loop search cost.

Matched efficiency comparison

The matched comparison asks whether temporal screening improves the cost of obtaining an instance that satisfies the same witness-validation rule. Host circuits, 500-attempt budgets, circuit–seed schedules, trigger-length and inter-phase-gap distributions, effective-literal range, payload-template mix, added-cell budget, reset semantics, replay requirements, post-generation audit depth and timeout, and attempt-level timeout policy are identical. All four flows use the same parser, trace-conditioned rare-event inventory, insertion backend, simulator, witness-validation gate, and resource logger. They differ in candidate construction and in whether a symbolic query is placed inside the candidate loop. SeqCube-HT uses the complete trace-backed construction and witness-validation flow defined in Section "Methodology"; the comparator-specific construction is defined below.

Candidate-first comparators Candidate-first+replay (CF-Replay) independently samples L rare events from the common inventory, assigns temporal gaps and a payload template, and proceeds directly to insertion and replay. Candidate-first+SAT/BMC (CF-SAT/BMC) is an author-defined diagnostic comparator rather than a reproduction of a published end-to-end generator. It uses the same sampler but first issues a reset-constrained symbolic query. The clean and candidate-infected Boolean transition relations are unrolled to the configured bound, selected trigger literals are asserted at their scheduled phases, and a payload miter requires a clean/infected difference at the recorded boundary. SAT supplies an

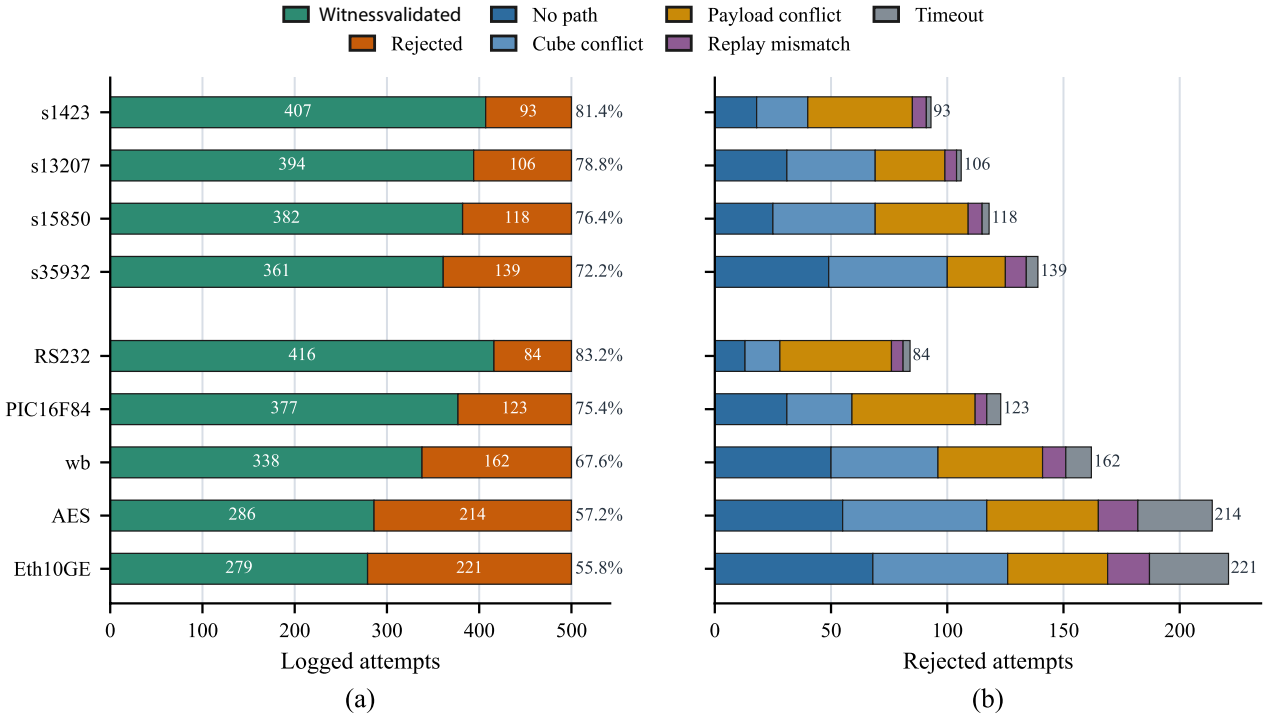


Fig. 6 Fixed-budget outcomes: **a** witness-validated and rejected attempts and **b** first failed stage

Table 3 Circuit scale and runtime (s per witness-validated instance)

Suite	Design	Instances	FF	Gates	Median	IQR	Replay	Audit
ISCAS'89	s1423	100	74	470	5.86	0.94	0.06	0.16
ISCAS'89	s13207	100	625	2108	11.72	1.68	0.19	0.39
ISCAS'89	s15850	100	513	2907	12.84	1.91	0.24	0.48
ISCAS'89	s35932	100	1728	6583	18.63	2.74	0.71	0.86
Extended IP	RS232	100	42	363	3.21	0.52	0.04	0.07
Extended IP	PIC16F84	100	381	2770	8.94	1.36	0.22	0.28
Extended IP	wb_conmax	100	770	83706	24.76	3.85	1.24	1.08
Extended IP	AES	100	6000	210382	39.42	6.71	2.76	2.35
Extended IP	EthMAC10GE	100	21830	80734	43.87	7.28	2.19	2.84

activating sequence for replay; UNSAT, unknown, or timeout rejects the candidate. Every invocation and status is logged. The paired inactive replay and common post-generation audit remain mandatory, so the inner-loop query does not weaken final witness validation.

Symbolic implementation CF-SAT/BMC uses Z3 through its Python Boolean API. For each candidate, the implementation constructs a fresh cycle-indexed unrolling and a fresh solver context; net/cycle expressions are memoized only within that query. Solver state, learned clauses, and unrolled prefixes are not reused across candidates. Resetless state is initialized under the all-zero analysis convention in Table 2, and clock, reset, scan, and test controls are held at their recorded normal-mode values after initialization. Calls are serialized in a single-worker run, no portfolio solver is used, and the 30-s timeout applies to each candidate query.

CG-Seq comparator (adapted) The published CG method constructs a PODEM-derived partial primary-input cube for each rare-value event, joins two events when their care bits can be merged without conflict, and uses a clique to represent events activatable by one common vector (Kumar et al. 2025). Our comparator retains this pairwise care-bit rule but adapts it to the present sequential protocol: candidate events are assigned to the sampled trigger phases, their partial assignments are

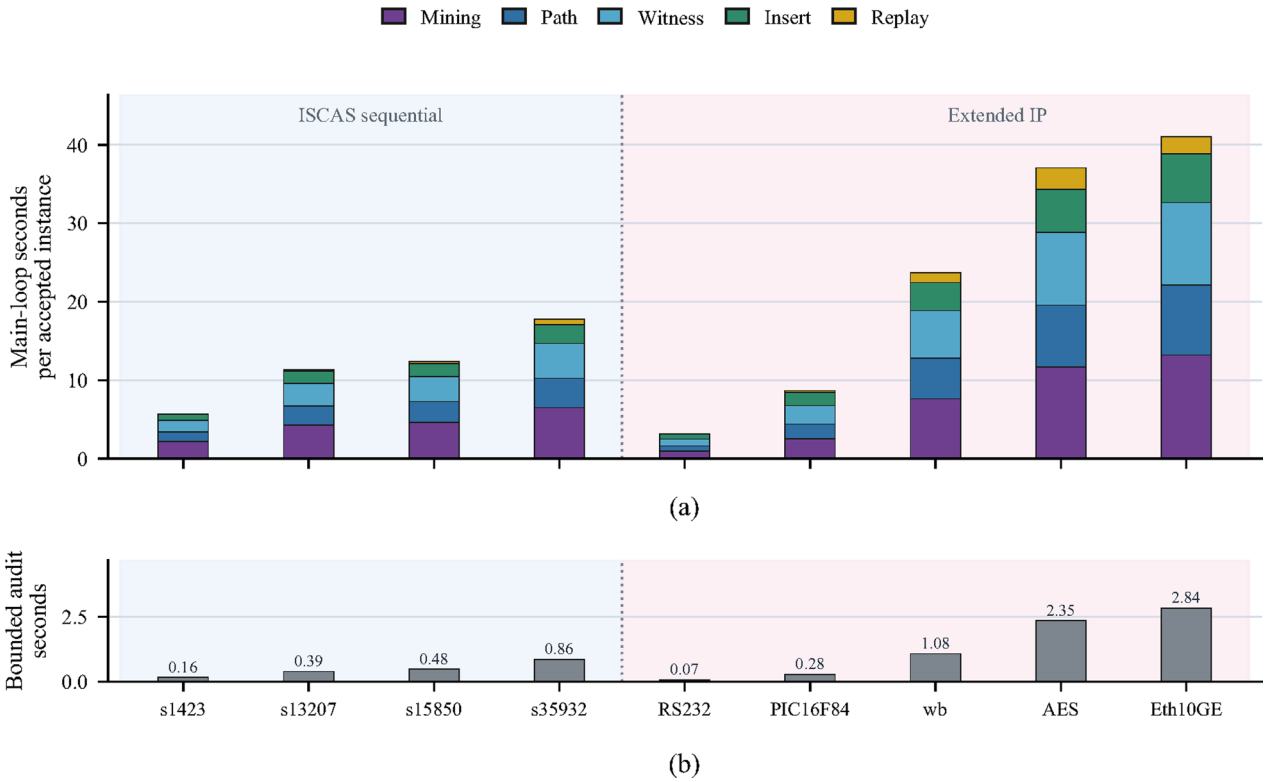


Fig. 7 Runtime decomposition: **a** construction and **b** bounded audit

placed in the corresponding unrolled input frames, and a mutually compatible size-L set is merged before the common completion, payload, replay, and audit stages. We denote it CG-Seq to distinguish this time-indexed adaptation from the published single-vector flow.

For every flow, wall-clock measurement begins before netlist normalization and event-inventory construction and ends after the final replay/audit outcome. Common preprocessing is charged independently; caches and intermediate candidates are not shared. Each run uses one worker process. Peak memory is the maximum resident set size of the process tree, and amortized seconds per witness-validated instance divides total elapsed time, including failed and timed-out attempts, by the number of instances satisfying Eq. (1). Per-circuit counts and resource diagnostics are retained before aggregation. All flows must pass active and inactive replay and the bounded audit; timeout or unknown is rejected.

SeqCube-HT obtains the highest witness-validation yield and the lowest amortized time under the matched protocol. The best-performing comparator implementation by yield is CG-Seq at 50.6%, compared with 72.0% for SeqCube-HT: a 21.4-point difference (paired run-level 95% CI: 19.3-−23.5). SeqCube-HT requires 20.9 s per witness-validated instance, or 52.9% of the CG-Seq cost of 39.5 s (paired ratio 0.529, 95% CI: 0.486−0.578).

The resource diagnostics explain part of this difference. Peak memory for SeqCube-HT is 4.8 GiB, within the 4.1−8.7 GiB range of the four flows. CF-SAT/BMC makes 4.6 inner-loop solver calls per witness-validated instance. The replay-rejection counts for CF-Replay, CF-SAT/BMC, CG-Seq, and SeqCube-HT are 1604, 238, 648, and 121; their timeout counts are 132, 594, 231, and 163. These counts are first-failure diagnostics rather than exhaustive rejection categories.

Table 4 compares four implemented flows under the stated protocol, not symbolic or compatibility-graph methods as classes. The values depend on the concrete solver and CG-Seq implementations: incremental CF-SAT/BMC with cross-candidate unrolling or solver-state reuse, or a different sequential CG construction, could change runtime and timeout-dependent yield. Our empirical claim is therefore restricted to the implementations and accounting boundary reported here.

Because all four flows receive the same trace-conditioned event inventory, Table 4 measures how efficiently each candidate-construction rule converts that inventory into witness-validated instances. The 72.0% witness-validation yield supports this construction-efficiency claim;

it is not an estimate of reachable-state coverage, temporal-behavior coverage, or diversity beyond the mined event pool.

Parameter sensitivity

A prespecified one-factor-at-a-time study tests whether the default settings lie in stable operating regions. The study uses s1423, s35932, and AES to cover small, medium, and large structures; non-varied parameters retain the values in Table 2. All five sweeps report witness-validation yield, late replay rejection, and runtime per witness-validated instance. Duplicate rate is reported for the four candidate-construction parameters, but not for audit depth because candidate signatures are fixed before the audit. Figure 8 summarizes the sweeps.

Macro-averaged values over the three designs show that the defaults avoid the steepest tradeoffs. A beam width of 64 lies on the yield plateau, whereas 128 adds 36% runtime for 0.4 percentage points of yield. Requiring two shared state literals reduces late replay rejection from 10.7 to 4.9% without the search-space contraction observed at three or four literals. Audit depths of 32 and 40 differ by 0.2 percentage points on this subset, indicating local saturation rather than a universal depth requirement. Longer triggers and stricter rarity thresholds reduce duplication and short-window exposure at the cost of lower yield and higher runtime; no circuit-specific optimum is selected.

Rarity and implementation footprint

Executability alone does not make an instance useful if ordinary random testing activates it readily. We therefore evaluate two properties together: rarity under unguided normal-mode stimulation and the amount of logic added to achieve that rarity.

Raw trigger width is a poor measure for temporal triggers because reset, scan, and other protocol-fixed literals do not contribute to accidental activation. We instead count *effective literals*: the non-frozen literals in the final merged trigger cube. The sweep varies this count from 2 to 30 in steps of 2 and tests 100 witness-validated instances for every design and setting.

Random detectability is measured at the instance level. For each infected instance, we start from reset and run $N = 100{,}000$ independent normal-mode random windows of length $W = 12$, holding clock, reset, scan-enable, test-mode, and other nonfunctional controls at their normal values. The trigger monitor is initialized at the first phase after reset, and an instance is counted as covered only when its temporal path completes inside the window. Trigger coverage (TC) is the percentage of the 100 instances at a given effective-literal setting that are activated by at least one window. TC is therefore a fixed-budget, fixed-window screening metric rather than an intrinsic trigger probability. In particular, a path whose temporal span exceeds W cannot be completed by this campaign, so comparisons involving such paths are jointly affected by literal count and temporal span.

Random coverage under the 12-cycle campaign falls within a circuit-dependent transition band rather than decreasing linearly (Fig. 9). For s1423 and s13207, observed coverage moves from near-complete to near-zero across the groups in the high teens and low twenties; s35932 remains detectable longer, while s15850 has the smoothest tail. The horizontal position and slope of each curve cannot be attributed to effective-literal count alone. Coverage can decrease because phase-local literals are harder to satisfy, because a larger fraction of the sampled paths has temporal span greater than $W = 12$, or both.

The Extended IP designs exhibit the same joint dependence on literal constraints and path span. RS232 loses observed coverage after relatively few effective constraints, PIC16F84 decays more slowly, wb_conmax and EthMAC10GE occupy the intermediate region, and AES has the longest tail. Cross-circuit differences may also reflect different temporal-span distributions. The curves therefore compare window-limited detectability under the fixed (N, W) campaign; they neither estimate nor rank intrinsic trigger-activation probabilities.

The decline in short-window coverage is not obtained by scaling the inserted logic in proportion to host size (Fig. 10). Normalized overhead is largest on RS232 and s1423 because similar absolute insertion budgets occupy a greater fraction of these small designs. Payload observation is also distributed across boundary types: some

Table 4 Matched implementation-level generation efficiency (4500 attempts)

Method	Attempts	Witness-validation yield (95% CI)	Amortized time s/instance (95% CI)	Peak GiB
CF-Replay	4500	30.8% (29.5–32.2)	58.4 (54.9–62.7)	4.1
CF-SAT/BMC	4500	39.4% (38.0–40.8)	86.7 (81.2–92.8)	8.7
CG-Seq	4500	50.6% (49.1–52.1)	39.5 (36.8–42.7)	5.3
SeqCube-HT	4500	72.0% (70.7–73.3)	20.9 (19.6–22.4)	4.8

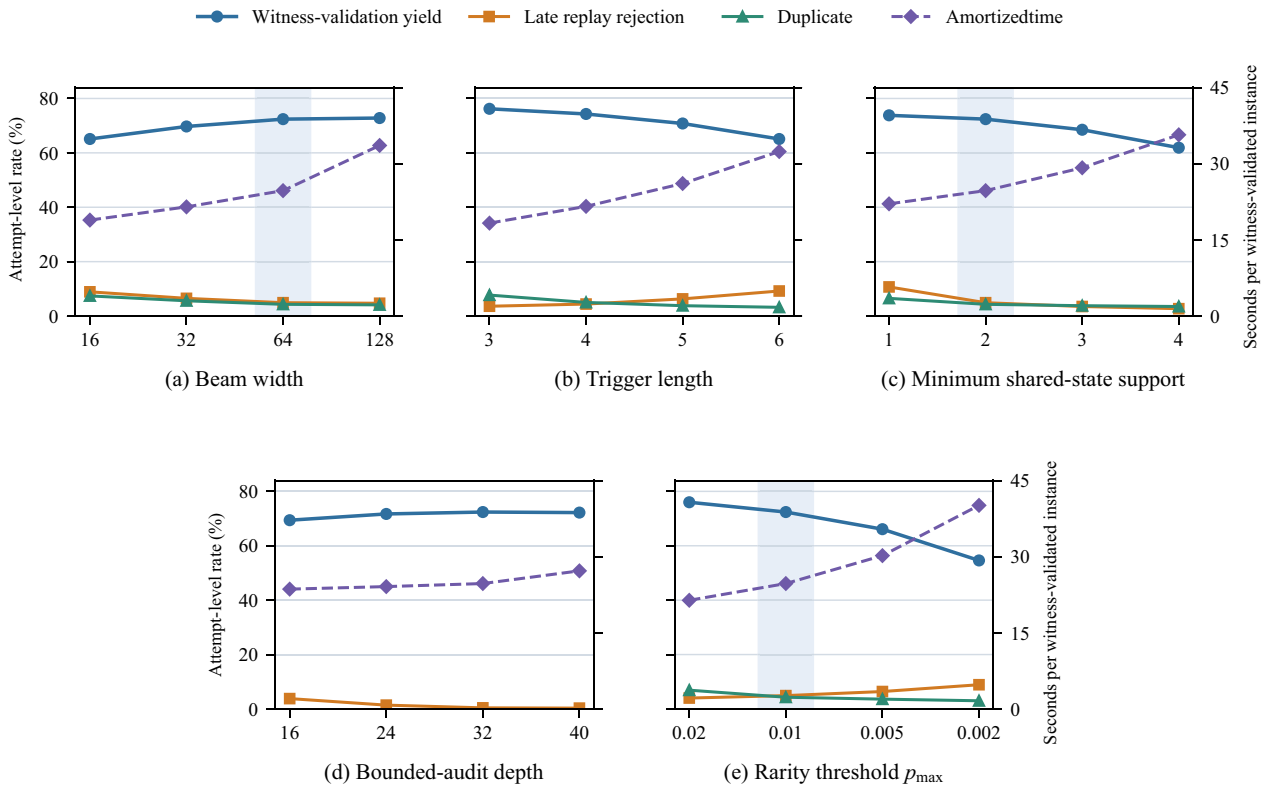


Fig. 8 One-factor-at-a-time sensitivity to five generation parameters. Shaded positions denote fixed scalar settings. Trigger length is sampled; audit depth is adaptive

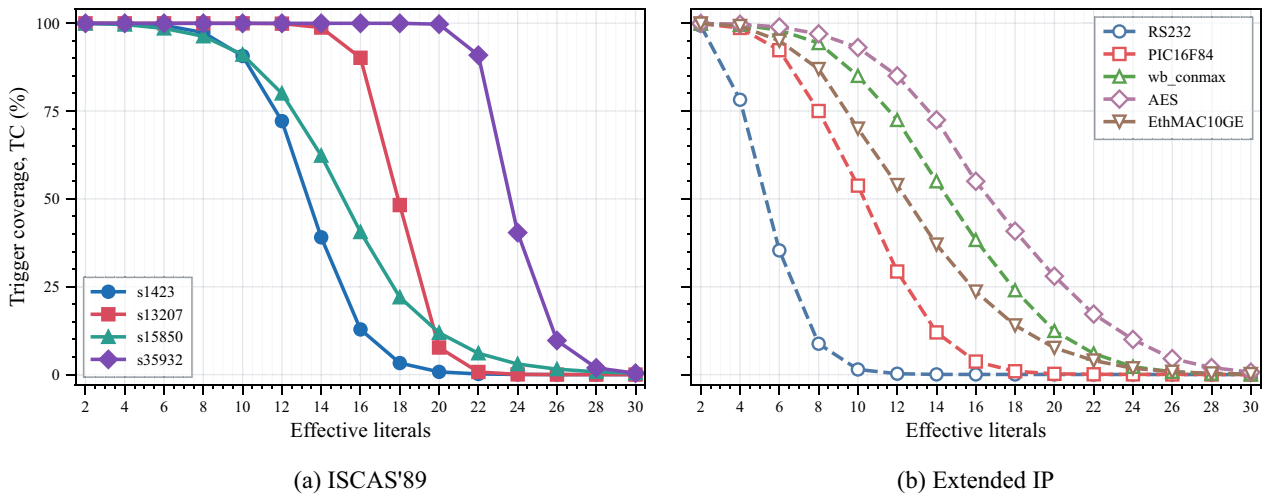


Fig. 9 Fixed-budget random-window trigger coverage versus effective-literal count ($N = 100{,}000, W = 12$). The curves jointly reflect literal constraints and the temporal-span distribution; paths with span greater than W cannot be covered by this campaign

effects reach primary outputs directly, while others cross a state boundary before becoming externally visible. Under the evaluated netlist model, the generated instances therefore combine low observed short-window coverage with a modest gate-count footprint.

Graph-detector characterization

Inventory-matched characterization

Three representative gate-level graph backbones serve as fixed structural probes: a GCN-style node classifier following recent localization studies (Xiao et al. 2025; Yasaei et al. 2022), a GAT-style edge-attention model motivated by GREAT (Li et al. 2025), and a GraphSAGE model following gate-level netlist detection work (Ma et al. 2025). Related graph-learning and netlist-sampling detectors further motivate this setting (Yasaei and Faruque 2025; Yasaei et al. 2021; Cheng et al. 2022, 2023; Lashen et al. 2023). The experiment asks whether insertion sources leave different structural evidence for the same probe family; it neither establishes detector universality nor tunes a model specifically against SeqCube-HT.

The inventory-matched comparison uses the four ISCAS'89 circuits shared by the reference corpora. Random denotes random insertion followed by the common witness gate, TRIT-TS is the available sequential-trigger subset of TRIT, CG contributes compatibility-graph artifacts, and SeqCube-HT contributes temporal state-cube instances. Here the label CG denotes the available CG artifact source; it is distinct from the time-indexed CG-Seq comparator implementation used only in Section "Matched efficiency comparison". Because TRIT-TS availability is uneven, each source supplies 51/100/100/63 infected instances on s1423/s13207/s15850/s35932, respectively, each paired with a clean graph. Sampling and split construction are repeated over ten seeds. This experiment matches host circuit and sample count; the later covariate-balanced protocol additionally controls measured insertion characteristics.

Every insertion source is exported to the same gate-level graph representation before training. Nodes are standard-cell or flip-flop instances, and directed edges follow driver-to-sink dependencies. Flip-flops are retained because the benchmarks contain sequential Trojans. The shared feature vector combines cell type with degree, primary-input/output distance, topological level, fanout, and harmonic centrality; numerical features are normalized per circuit. Instance names, net names, source names, and insertion-method identifiers are excluded. Node labels cover only inserted or structurally modified cells, including trigger monitors, sticky logic, payload mux/override/bit-flip logic, and inserted glue. Unmodified host gates on replay, sensitization, or propagation paths remain unlabeled.

The three split protocols separate within-source, cross-circuit, and cross-source behavior. The instance-random setting uses stratified 70/15/15 partitions within each host and source. Circuit-held-out testing leaves one ISCAS'89 host out, whereas source-held-out testing leaves one insertion surface out. The SeqCube-HT source-held-out result therefore trains on Random, TRIT-TS, and CG before testing on SeqCube-HT. The

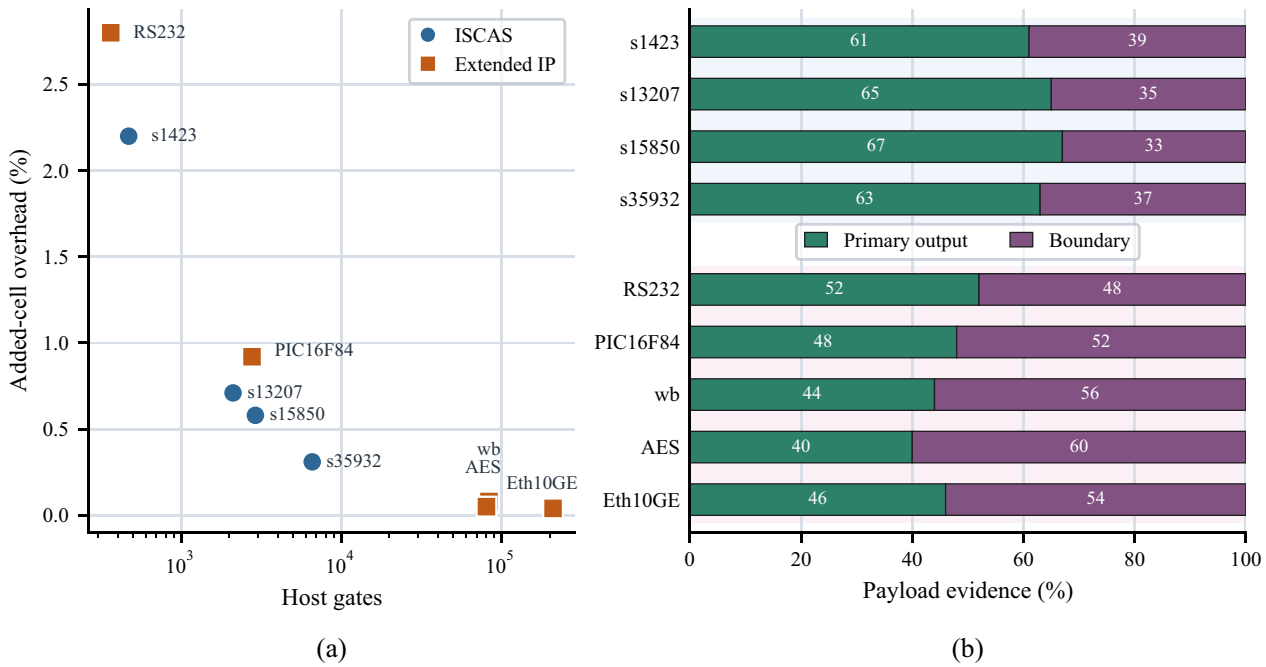


Fig. 10 Implementation footprint: **a** added-cell overhead and **b** observation boundary.

maximum node probability defines the graph score (Table 5). We report graph-level F1 and node-level localization recall and use clean-class specificity as a negative-class sanity check.

Under inventory matching, the probes separate SeqCube-HT from the three reference sources (Fig. 11). Random, TRIT-TS, and CG obtain mean F1 scores of 95.3%, 92.5%, and 90.5%, with localization recall of 91.0%, 88.0%, and 84.1%. The three-backbone average for SeqCube-HT is 69.2% F1 and 55.7% localization recall. Because this comparison controls only host and sample count, the next experiment tests whether measured insertion covariates account for the gap.

The localization ordering is not caused by a single host circuit. SeqCube-HT has lower recall than the reference sources on s1423, s13207, s15850, and s35932, although the size of the gap varies by host (Fig. 12). This variation motivates controlling both circuit and insertion characteristics before interpreting source effects.

Transfer across insertion sources is more difficult than within-source evaluation for every corpus (Fig. 13). For SeqCube-HT, F1 decreases from 69.2% under instance-random testing to 47.8% when its source is held out; localization recall decreases from 55.7% to 31.4%. Circuit-held-out results lie between the within-source and source-held-out settings. The persistent ordering

Table 5 Graph-detector protocol

Parameter	Setting
Nodes	Standard cells and flip-flops
Edges	Directed driver-to-sink dependencies
Node features	Cell type, degree, PI/PO distance, level, fanout, and harmonic centrality
Excluded features	Instance names, net names, source names, and insertion-method identifiers
Node labels	Inserted or structurally modified cells
Graph score	$s_G = \max_{v \in V} p_v$
Optimizer	Adam, learning rate 0.001
Loss	Weighted cross entropy
Training limit	200 epochs with early stopping
Splits	70/15/15 instance-random; circuit-held-out; source-held-out
Matching	Circuit/source counts with paired clean graphs
Repeats	10 seeded sampling and split runs
Aggregation	Circuit-macro scores

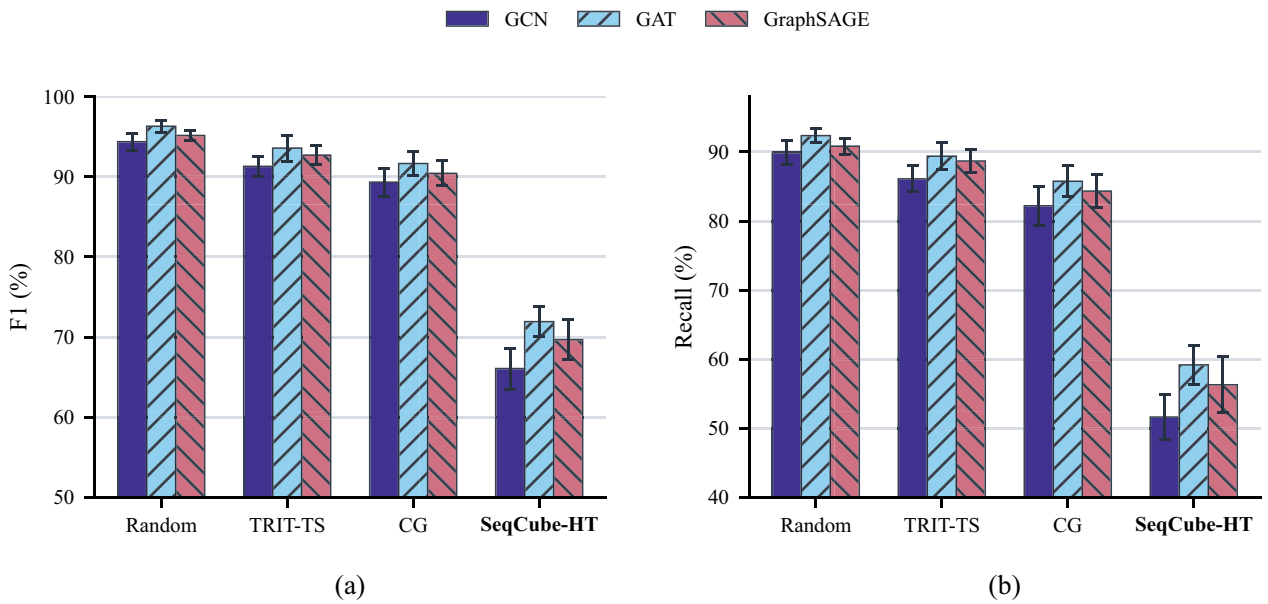


Fig. 11 Inventory-matched performance: **a** detection F1 and **b** localization recall; error bars show across-circuit SD.

indicates a source shift, but does not by itself identify which structural covariates cause it.

Covariate-Balanced Characterization

We therefore repeat the comparison on the covariate-balanced corpus to test whether the ordering remains after the major measured structural differences are controlled.

Within each host circuit and payload-observation type, exact matching first aligns payload template and trigger-length bin across sources. Nearest-neighbor matching then uses inserted/modified-cell count, Trojan-node ratio, temporal span, PI/PO distance, insertion depth, and local fanout. Detector outcomes are not used. The simultaneous match retains 216 infected instances per source (68.8% of the common inventory) and their paired clean graphs. Continuous covariates are standardized separately; categorical variables use binary indicators. Table 6 reports the largest absolute pairwise standardized mean difference (SMD) across sources and variables in each row.

Covariate matching narrows, but does not remove, the difference between SeqCube-HT and the reference sources (Fig. 14). SeqCube-HT increases from 69.2% to 77.2% F1 and from 55.7% to 66.1% localization recall. Relative to CG, the closest reference source, the remaining differences are 9.0 F1 points (seed-paired 95% CI: 6.6−11.4; Holm-adjusted $p < 0.001$) and 12.3 localization-recall points (9.1−15.6; Holm-adjusted $p < 0.001$). The adjusted values belong to separate prespecified Holm families. They quantify seeded-run variability on the fixed four-circuit corpus. The residual differences may reflect unmeasured structural covariates or insertion-style distribution shift; they do not establish intrinsic stealth or physical realism.

For SeqCube-HT, graph-level precision, recall, specificity, F1, and PR-AUC are 78.6% (76.7−80.4), 75.9% (73.8−77.9), 82.8% (80.9−84.6), 77.2% (75.4−78.9), and 82.9% (81.3−84.5), respectively. Localization precision is 62.4% (59.7−65.0), and localization recall is 66.1% (63.4−68.7). Parenthesized ranges are 95% Student intervals over the ten seed-level macro scores.

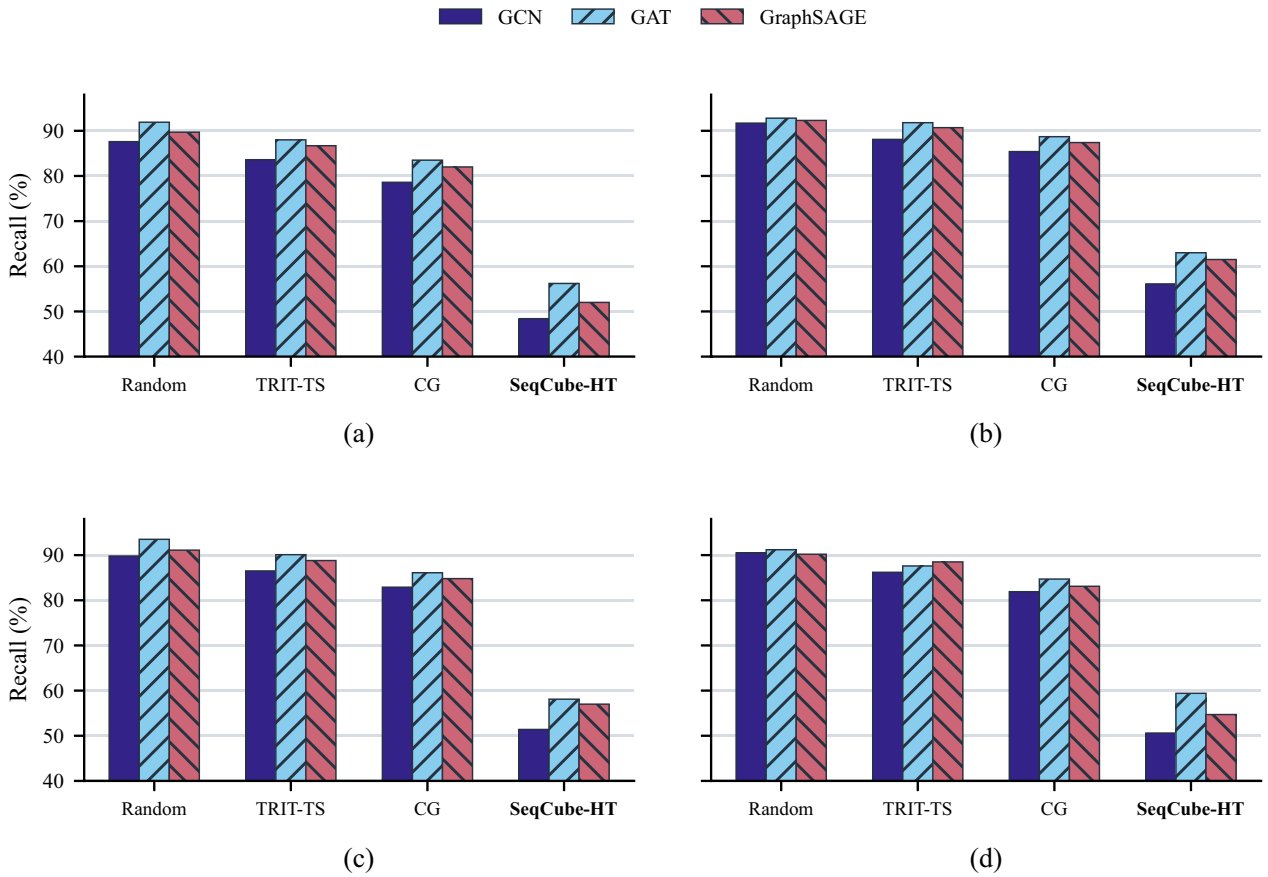


Fig. 12 Per-circuit localization recall under inventory matching: **a** s1423, **b** s13207, **c** s15850, and **d** s35932

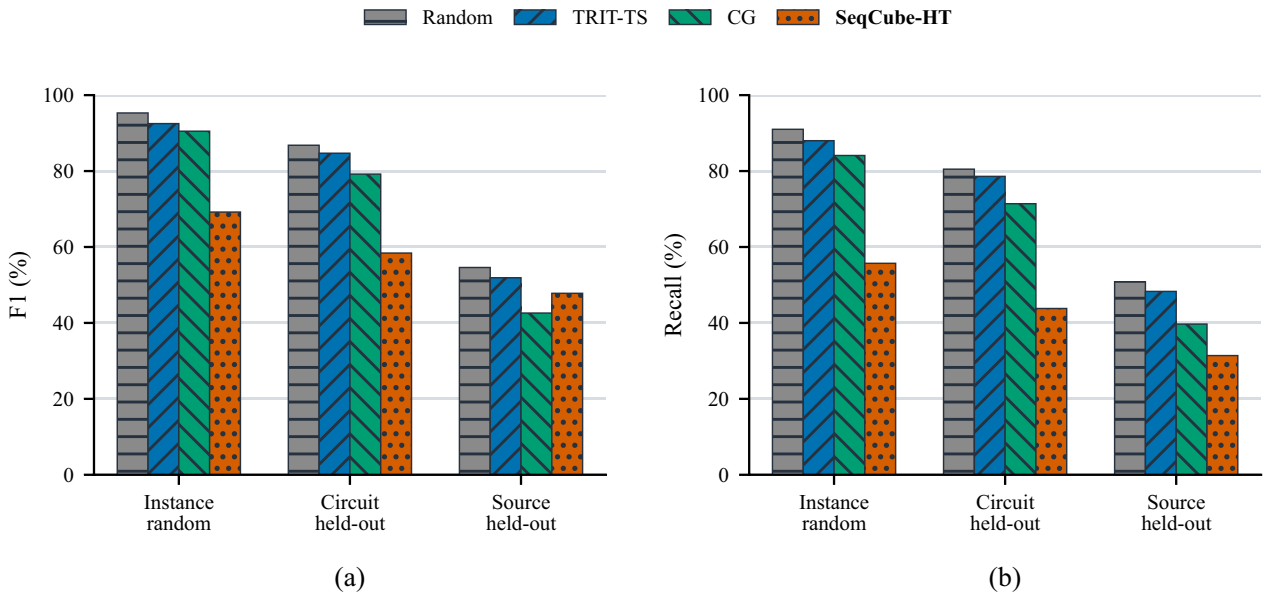


Fig. 13 Held-out performance: **a** detection F1 and **b** localization recall

Table 6 Covariate balance before and after matching (maximum pairwise SMD per group)

Covariate group	Before matching max |SMD|	After matching max |SMD|
Inserted/modified cells	0.68	0.07
Trojan-node ratio	0.74	0.06
Trigger length; temporal span	0.81	0.08
Payload template; observation-boundary type	0.46	0.00
Insertion depth; PI distance; PO distance	0.59	0.09
Local fanout	0.44	0.05

Extended-IP detector and cross-scale evaluation

The Extended-IP experiment tests whether the same detector protocol can be applied beyond the four ISCAS'89 hosts. It includes Random and the author-defined CF-SAT/BMC comparator after applying the common replay-and-audit validation rule, together with SeqCube-HT; these are the three sources with complete retained corpora on RS232, PIC16F84, wb_conmax, AES, and EthMAC10GE. For each IP and source, 80 infected graphs and their paired clean graphs are evaluated over ten seeds. We report within-IP stratified results and cross-scale transfer trained on ISCAS'89 and tested on Extended IP without threshold retuning. The first

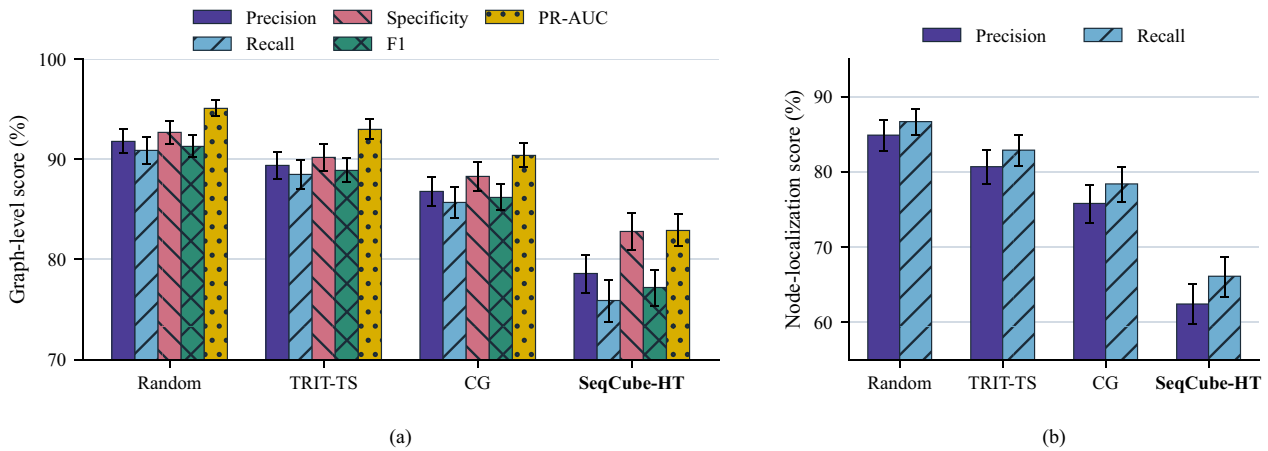


Fig. 14 Covariate-balanced performance: **a** graph detection and **b** node localization; error bars show 95% CIs over the ten seed-level macro scores

remains a within-host estimate; the second combines insertion-source shift with a substantial graph-size shift.

Within the five Extended IPs, SeqCube-HT obtains 73.6% F1 (95% CI: 70.9–76.2), 79.8% PR-AUC, and 60.9% localization recall (Fig. 15). Without threshold retuning, ISCAS-to-IP transfer decreases to 56.8% F1 (53.4–60.1), 61.9% PR-AUC, and 40.7% localization recall. The drop characterizes the combined insertion-source and graph-scale shift under a fixed decision threshold; it is not attributed to either factor alone.

Component ablation

The final experiment asks which constraints prevent late replay and observability failures. On a fixed subset, we compare rare-event-only selection, state-cube path stitching, trigger paths with payload-witness constraints, and the complete trigger–payload flow.

Rare-event-only selection has low yield and poor observation because rarity does not imply temporal executability. State-cube paths raise the replay pass rate by screening direct endpoint conflicts, and payload-witness constraints remove candidates without a sensitized observation path (Fig. 16). The complete flow reaches 71.8% witness-validation yield, 100.0% replay and observability pass rates, 24.7 s per witness-validated instance, and a 4.1% duplicate rate. Its lower runtime and duplication follow from fewer late failures and explicit signature suppression.

Conclusion

Sequential HT benchmarks are useful for detector evaluation only when their triggers execute from a defined reset state and their payload effects reach an observation boundary. SeqCube-HT incorporates this

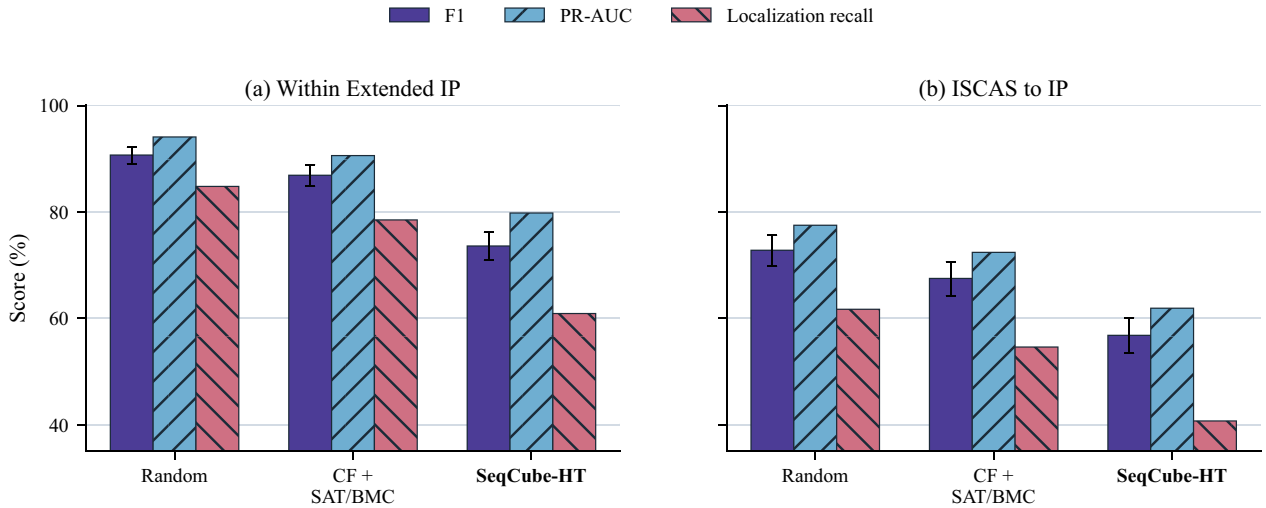


Fig. 15 Extended-IP performance: **a** within-IP and **b** ISCAS-to-IP transfer; error bars show F1 95% CIs. CF + SAT/BMC denotes the author-defined comparator after the common replay-and-audit validation rule

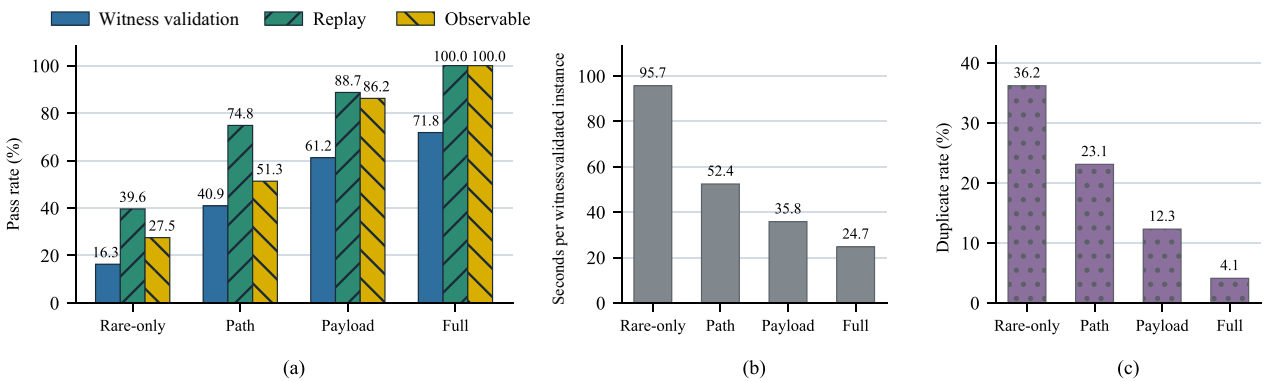


Fig. 16 Component ablation: **a** pass rates, **b** runtime, and **c** duplicate rate.

evidence into candidate construction by organizing trace-backed state transitions into directed paths, completing those paths into concrete inputs, and coupling trigger selection with payload sensitization. Active and inactive replay and a bounded audit determine witness validation, while the records retained for each evaluated instance support package-level replay and label reconstruction.

Across nine designs, SeqCube-HT produced 3,240 witness-validated instances from 4,500 attempts. Under the matched protocol, it achieved 72.0% witness-validation yield and 20.9 s per witness-validated instance, compared with 50.6% and 39.5 s for the best-performing comparator implementation. Covariate matching narrowed but did not remove the differences observed by the graph-detector probes, indicating that the generated corpus retains source-specific structure beyond the measured insertion covariates. The Extended IP experiments further established computational feasibility on the evaluated larger gate-level netlists. Together, these results support trace-backed temporal construction as an efficient basis for witness-validated sequential HT benchmarks within the evaluated trace budget and validation protocol. Reachable-state coverage and diversity beyond the mined event pool remain outside the evaluated question. Future work will extend the current single-clock gate-level model to multi-clock and physical-design-aware settings.

Acknowledgements
Not applicable.

Author contributions
Yuhang Xu contributed to the study design, method development, experiments, data analysis, and manuscript drafting. Wei Guo supervised the work, contributed to the study design, and revised the manuscript. Wenbo Zhang, Shuaikang Hou, Wenjian Zhang, and Zhihao Xu contributed to method discussion, experimental analysis, and manuscript revision. All authors read and approved the final manuscript.

Funding
This work was funded by the National Key Research and Development Program of China (Grant No. 2022YFB2901000).

Data availability
The data generated and analyzed during the current study are available from the corresponding author on reasonable request.

Declarations

Competing interests
The authors declare that they have no competing interests.

Received: 25 June 2026 Accepted: 27 September 2026
Published online: 09 October 2026

References

Becker GT, Regazzoni F, Paar C, Burleson WP (2013) Stealthy dopant-level hardware trojans. In: Proceedings of the international workshop on cryptographic hardware and embedded systems, pp 197–214. https://doi.org/10.1007/978-3-642-40349-1_12

Bhunia S, Hsiao MS, Banga M, Narasimhan S (2014) Hardware Trojan attacks: threat analysis and countermeasures. Proc IEEE 102(8):1229–1247. https://doi.org/10.1109/JPROC.2014.2334493

Bhunia S, Huang Y, Mishra P (2016) MERS: statistical test generation for side-channel analysis based Trojan detection. In: Proceedings of the ACM SIGSAC conference on computer and communications security, pp 130–141. https://doi.org/10.1145/2976749.2978396

Brglez F, Bryan D, Kozminski K (1989) Combinational profiles of sequential benchmark circuits. In: Proceedings of the IEEE international symposium on circuits and systems, pp 1929–1934. https://doi.org/10.1109/ISCAS.1989.100747

Brglez F, Fujiwara H (1985) A neutral netlist of 10 combinational benchmark circuits and a target translator in FORTRAN. In: Proceedings of the IEEE international symposium on circuits and systems

Chakraborty RS, Wolff F, Paul S, Papachristou C, Bhunia S (2009) MERO: a statistical approach for hardware Trojan detection. In: Proceedings of the international workshop on cryptographic hardware and embedded systems, pp 396–410. https://doi.org/10.1007/978-3-642-04138-9_28

Chen L, Dong C, Wu Q, Liu X, Guo X, Chen Z, Zhang H, Yang Y (2025) GNN4HT: a two-stage GNN-based approach for hardware Trojan multifunctional classification. IEEE Trans Comput Aided Des Integr Circuits Syst 44(1):172–185. https://doi.org/10.1109/TCAD.2024.3428469

Cheng D, Dong C, He W, Chen Z, Liu X, Zhang H (2023) A fine-grained detection method for gate-level hardware Trojan based on bidirectional graph neural networks. J King Saud Univ Comput Inf Sci 35(10):101822. https://doi.org/10.1016/j.jksuci.2023.101822

Cheng D, Dong C, He W, Chen Z, Xu Y (2022) GNN4Gate: A bi-directional graph neural network for gate-level hardware Trojan detection. In: Proceedings of the design, automation & test in Europe conference, pp 1315–1320. https://past.date-conference.com/proceedings-archive/2022/html/0717.html

Cruz J, Gaikwad P, Nair A, Chakraborty P, Bhunia S (2022) A machine learning based automatic hardware Trojan attack space exploration and benchmarking framework. In: Proceedings of the Asian hardware oriented security and trust symposium. https://doi.org/10.1109/AsianHOST56390.2022.10022234

Cruz J, Huang Y, Mishra P, Bhunia S (2018) An automated configurable Trojan insertion framework for dynamic trust benchmarks. In: Proceedings of the design, automation & test in Europe conference, pp 1598–1603. https://doi.org/10.23919/DATE.2018.8342270

Fyrbiak M, Wallat S, Swierczynski P, Hoffmann M, Hoppach S, Wilhelm M, Weidlich T, Tessier R, Paar C (2019) HAL-the missing piece of the puzzle for hardware reverse engineering, Trojan detection and insertion. IEEE Trans Dependable Secure Comput 16(3):498–510. https://doi.org/10.1109/TDSC.2018.2812183

Gohil V, Guo H, Patnaik S, Rajendran J (2022) Attrition: attacking static hardware Trojan detection techniques using reinforcement learning. In: Proceedings of the ACM SIGSAC conference on computer and communications security, pp 1275–1289. https://doi.org/10.1145/3548606.3560690

Goldstein LH, Thigpen EL (1980) SCOAP: Sandia controllability/observability analysis program. In: Proceedings of the design automation conference, pp 190–196. https://doi.org/10.1145/800139.804528

Hasegawa K, Yanagisawa M, Togawa N (2017) Trojan-feature extraction at gate-level netlists and its application to hardware-Trojan detection using random forest classifier. In: Proceedings of the IEEE international symposium on circuits and systems, pp 1–4. https://doi.org/10.1109/ISCAS.2017.8050827

Hepp A, Perez T, Pagliarini S, Sigl G (2022) A pragmatic methodology for blind hardware Trojan insertion in finalized layouts. In: Proceedings of the IEEE/ACM international conference on computer-aided design, pp 1–9. https://doi.org/10.1145/3508352.3549452

Hicks M, Finnicum M, King ST, Martin MMK, Smith JM (2010) Overcoming an untrusted computing base: detecting and removing malicious hardware automatically. In: Proceedings of the IEEE symposium on security and privacy, pp 159–172. https://doi.org/10.1109/SP.2010.18

Hu W, Oberg J, Tiwari M, Sherwood T, Kastner R (2014) Gate-level information flow tracking for security lattices. ACM Trans Design Autom Electron Syst 20(1):1–25. https://doi.org/10.1145/2676548

Jyothi V, Krishnamurthy P, Khorrami F, Karri R (2017) TAINT: tool for automated insertion of trojans. In: Proceedings of the IEEE international conference on computer design, pp 545–548. https://doi.org/10.1109/ICCD.2017.95
Krieg C (2023) Reflections on trusting Trust-Hub. In: Proceedings of the IEEE/ACM International conference on computer-aided design, pp 1–9. https://doi.org/10.1109/ICCAD57390.2023.10323782
Kumar G, Shaik A.H, Riaz A, Prasad Y, Ahlawat S (2025) Compatibility graph assisted automatic hardware Trojan insertion framework. In: Proceedings of the design, automation & test in Europe conference, pp 1–7. https://doi.org/10.23919/DATE64628.2025.10992945
Lashen H, Alrahis L, Knechtel J, Sinanoglu O (2023) TrojanSAINT: gate-level netlist sampling-based inductive learning for hardware Trojan detection. In: Proceedings of the IEEE international symposium on circuits and systems, pp 1–5. https://doi.org/10.1109/ISCAS46773.2023.10181403
Li Z, Cheng W, Tang H, Wang Y (2025) GREAT: Global representation and edge-attention for hardware Trojan detection. In: Proceedings of the 55th Annual IEEE/IFIP international conference on dependable systems and networks, pp 233–245. https://doi.org/10.1109/DSN64029.2025.00034
Lyu Y, Mishra P (2021) Scalable activation of rare triggers in hardware trojans by repeated maximal clique sampling. IEEE Trans Comput Aided Des Integr Circuits Syst 40(7):1287–1300. https://doi.org/10.1109/TCAD.2020.3019984
Lyu Y, Mishra P (2021) MaxSense: side-channel sensitivity maximization for Trojan detection using statistical test patterns. ACM Trans Design Autom Electron Syst 26(3):1–21. https://doi.org/10.1145/3436820
Ma P, Li J, Liu H, Shi J, Zhang S, Pan W, Hao Y (2025) Hardware Trojan detection methods for gate-level netlists based on graph neural networks. IEEE Trans Comput 74(5):1470–1481. https://doi.org/10.1109/TC.2025.3533085
Mishra P, Bhunia S, Tehranipoor M (eds) (2017) Hardware IP security and trust. Springer, Cham. https://doi.org/10.1007/978-3-319-49025-0
Nahiyan A, Sadi M, Vittal R, Contreras G, Forte D, Tehranipoor M (2017) Hardware Trojan detection through information flow security verification. In: Proceedings of the IEEE international test conference. https://doi.org/10.1109/TEST.2017.8242062
Nozawa K, Hasegawa K, Hidano S, Kiyomoto S, Hashimoto K, Togawa N (2021) Generating adversarial examples for hardware-Trojan detection at gate-level netlists. J Inf Process 29:236–246. https://doi.org/10.2197/ipsjjip.29.236
Pan Z, Mishra P (2021) Automated test generation for hardware Trojan detection using reinforcement learning. In: Proceedings of the Asia and South Pacific design automation conference, pp 408–413. https://doi.org/10.1145/3394885.3431595
Perez T, Pagliarini S (2023) Hardware Trojan insertion in finalized layouts: from methodology to a silicon demonstration. IEEE Trans Comput Aided Des Integr Circuits Syst 42(7):2094–2107. https://doi.org/10.1109/TCAD.2022.3223846
Popryho Y, Pal D, Partin-Vaisband I (2026) Automated hardware Trojan insertion in industrial-scale designs. In: Proceedings of the design, automation & test in Europe conference, pp 1–7. https://past.date-conference.com/proceedings-archive/2026/DATA/1294.pdf
Rostami M, Koushanfar F, Karri R (2014) A primer on hardware security: models, methods, and metrics. Proc IEEE 102(8):1283–1295. https://doi.org/10.1109/JPROC.2014.2335155
Saha S, Chakraborty RS, Nuthakki SS, Anshul, Mukhopadhyay D (2015) Improved test pattern generation for hardware Trojan detection using genetic algorithm and boolean satisfiability. In: Proceedings of the international workshop on cryptographic hardware and embedded systems, pp 577–596.https://doi.org/10.1007/978-3-662-48324-4_29
Salmani H (2017) COTD: reference-free hardware Trojan detection and recovery based on controllability and observability in gate-level netlist. IEEE Trans Inf Forensics Secur 12(2):338–350. https://doi.org/10.1109/TIFS.2016.2613842
Salmani H, Tehranipoor M, Karri R (2013) On design vulnerability analysis and trust benchmarks development. In: Proceedings of the IEEE international conference on computer design, pp 471–474. https://doi.org/10.1109/ICCD.2013.6657085
Sarihi A, Patooghy A, Jamieson P, Badawy A-HA (2024) Trojan playground: a reinforcement learning framework for hardware Trojan insertion and detection. J Supercomput 80:14295–14329. https://doi.org/10.1007/s11227-024-05963-8
Shakya B, He T, Salmani H, Forte D, Bhunia S, Tehranipoor M (2017) Benchmarking of hardware trojans and maliciously affected circuits. J Hardw Syst Secur 1(1):85–102. https://doi.org/10.1007/s41635-017-0001-6
Sturton C, Hicks M, Wagner D, King ST (2011) Defeating UCI: building stealthy and malicious hardware. In: Proceedings of the IEEE symposium on security and privacy, pp 64–77.https://doi.org/10.1109/SP.2011.32
Su T, Wang Y, Xu S, Zhang L, Feng S, Song J, Liu Y, Tang Y, Zhang Y, Li S, Guo Y, Liu H (2024) Improving the ability of thermal radiation based hardware Trojan detection. In: Proceedings of the 33rd USENIX security symposium, pp 127–144. https://www.usenix.org/conference/usenixsecurity24/presentation/su-ting
Tehranipoor M, Koushanfar F (2010) A survey of hardware Trojan taxonomy and detection. IEEE Design Test Comput 27(1):10–25. https://doi.org/10.1109/MDT.2010.7
Tiwari M, Li X, Wassel HMG, Chong FT, Sherwood T (2009) Complete information flow tracking from the gates up. In: Proceedings of the international conference on architectural support for programming languages and operating systems, pp 109–120. https://doi.org/10.1145/1508244.1508258
Trippel T, Shin K.G, Bush KB, Hicks M (2021) Bomberman: defining and defeating hardware ticking timebombs at design-time. In: Proceedings of the IEEE symposium on security and privacy, pp 970–986. https://doi.org/10.1109/SP40001.2021.00052
Trippel T, Shin K.G, Bush K.B, Hicks M (2020) ICAS: An extensible framework for estimating the susceptibility of IC layouts to additive trojans. In: Proceedings of the IEEE symposium on security and privacy, pp 1742–1759. https://doi.org/10.1109/SP40000.2020.00083
U.S. Department of Defense: securing defense-critical supply Chains: an action plan developed in response to President Biden's executive order 14017. U.S. Department of Defense report (2022). https://www.businessdefense.gov/docs/eo/DoD-EO14017-Report_Securing_Defense-Critical-Supply-Chains.pdf
Waksman A, Suozzo M, Sethumadhavan S (2013) FANCI: Identification of stealthy malicious logic using boolean functional analysis. In: Proceedings of the ACM SIGSAC conference on computer and communications security, pp 697–708. https://doi.org/10.1145/2508859.2516654
Xiao J, Chai S, Gao Y, Huang Y, Zhang F, Chen T (2025) HTs-GCN: identifying hardware Trojan nodes in integrated circuits using a graph convolutional network. IEEE Trans Comput Aided Des Integr Circuits Syst 44(6):2353–2366. https://doi.org/10.1109/TCAD.2024.3520522
Yasaei R, Faezi S, Faruque MAA (2022) Golden reference-free hardware Trojan localization using graph convolutional network. IEEE Trans Very Large Scale Integr Syst 30(10):1401–1411. https://doi.org/10.1109/TVLSI.2022.3191683
Yasaei R, Chen L, Yu SY, Al Faruque MA (2025) Hardware Trojan detection using graph neural networks. IEEE Trans Comput Aided Des Integr Circuits Syst 44(1):25–38. https://doi.org/10.1109/TCAD.2022.3178355
Yu S, Liu W, O'Neill M (2019) An improved automatic hardware Trojan generation platform. In: Proceedings of the IEEE computer society annual symposium on VLSI, pp 302–307. https://doi.org/10.1109/ISVLSI.2019.00062
Yu S.-Y, Yasaei R, Zhou Q, Nguyen T, Faruque MAA (2021) HW2VEC: A graph learning tool for automating hardware security. In: Proceedings of the IEEE international symposium on hardware oriented security and trust, pp 13–23. https://doi.org/10.1109/HOST49136.2021.9702281
Zhang J, Yuan F, Wei L, Liu Y, Xu Q (2015) VeriTrust: verification for hardware trust. IEEE Trans Comput Aided Des Integr Circuits Syst 34(7):1148–1161. https://doi.org/10.1109/TCAD.2015.2422836
Zhang J, Yuan F, Xu Q (2014) DeTrust: Defeating hardware trust verification with stealthy implicitly-triggered hardware trojans. In: Proceedings of the ACM SIGSAC conference on computer and communications security, pp 153–166. https://doi.org/10.1145/2660267.2660289

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.